UPDATED FEATURE TRACE

FootPath Cebu - Deep Code Tracing

Verified execution paths, authorization boundaries, database traces, failure handling, and defense-ready call chains.

Revision	Merged match-performance tracking
Source commit	db9e869 - feat: add secure match performance tracking
Verified suites	271 Django tests | 250 Flutter tests | clean Flutter analysis | no migration drift
Primary audience	Capstone defense, code review, and implementation tracing

Contents

This execution map follows only verified repository connections. Approximate locations use symbol names/nearby line numbers because line numbers can move. Django's default table names are stated where no explicit `db_table` overrides exist.

Code execution map index

Required map	Location in this document
Application startup trace	Workflow 1
Authentication trace	Workflows 2, 4, 5, and Authorization Trace
Navigation trace	Workflow 7 and Screen-to-Screen Navigation Traces
Role authorization trace	Workflow 6 and Authorization Trace
Player workflow trace	Workflows 8, 9, 16, 18, 25, 26, 28, 32
Coach workflow trace	Workflows 13-17, 27, 30-32 and Button-to-Database Traces
Scout workflow trace	Workflows 10-12: `NOT IMPLEMENTED IN CURRENT REPOSITORY`
Scouting-report trace	Workflows 10-12: `NOT IMPLEMENTED IN CURRENT REPOSITORY`
Training trace	Workflows 13, 14, and 28
Attendance trace	Workflow 14 and attendance lifecycle/query/variable traces
Performance trace	Workflows 15-17 and 30-32
Academic eligibility trace	Workflows 18-19
AI trace	Workflow 23: `NOT IMPLEMENTED IN CURRENT REPOSITORY`
File-upload trace	Workflow 21
Database query traces	Database Query Traces section
Variable traces	Variable Traces section
Object/model traces	Object / Model Traces section
Async traces	Async Code Traces section
Failure/error traces	Failure / Error Path Matrix and workflow failure paths
Top 20 important function call chains	Top 20 Important Function Call Chains section
Shared HTTP/offline-read trace	Cross-Cutting Trace: Shared Authenticated API Client and Owner-Scoped Cache
Production hardening trace	Repository Production Hardening Trace

Workflow 1: Application Startup

Trigger

The OS launches Flutter and invokes `main()` in `footpath_cebu/lib/main.dart` near line 15.

Step 1 - UI

There is no user input yet. `WidgetsFlutterBinding.ensureInitialized()` prepares plugins; `Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)` runs. `runApp(ProviderScope(child: FootPathApp(...)))` creates the widget tree.

Step 2 - State / Controller Layer

`ProviderScope` enables Riverpod. `FootPathApp.build()` creates `MaterialApp`; its `home` is `SessionBootstrapScreen` unless Firebase setup threw, in which case `_SetupErrorScreen` displays the caught message.

Step 3 - Service / Repository Layer

Startup uses Firebase Core and, in live mode, `_FootPathAppState.initState()` subscribes to foreground messages, opened-message events, and FCM token refresh. Session work is deferred to `SessionBootstrapScreen`. Dependency factories live in `core/di/providers.dart` and are lazily read.

Step 4 - Backend / API

No backend request is made by `main()`. The first API request occurs during session restoration.

Step 5 - Database

No database record is touched by `main()`.

Step 6 - Backend Response

Not applicable. Firebase initialization either completes or throws locally/plugin-side.

Step 7 - Model Conversion

No domain model conversion occurs.

Step 8 - State Update

The caught setup error becomes the immutable `FootPathApp.setupError` constructor value. Riverpod state begins inside `ProviderScope`; successful live startup retains the three messaging subscriptions until `dispose()` cancels them.

Step 9 - UI Update

Flutter renders `SessionBootstrapScreen` or `_SetupErrorScreen` through normal widget build.

Call chain: `main()` -> `ensureInitialized()` -> `Firebase.initializeApp()` -> `runApp()` -> `ProviderScope` -> `_FootPathAppState.initState()` -> FCM foreground/open/token listeners -> `FootPathApp.build()` -> `MaterialApp` -> `SessionBootstrapScreen`.

Failure path: Firebase initialization exception -> `setupError` string -> `_SetupErrorScreen`; no login/profile call is attempted.

Workflow 2: Authentication / Session Restoration

Trigger

`_SessionBootstrapScreenState.initState()` in `presentation/screens/session_bootstrap_screen.dart` schedules `_restore()` after widget initialization.

Step 1 - UI

`SessionBootstrapScreen` initially renders a progress indicator. `_restore()` calls `ref.read(restoreSessionProvider)()`.

Step 2 - State / Controller Layer

`restoreSessionProvider` exposes the `RestoreSession` use case from `core/di/providers.dart`. The screen awaits `UserProfile?`; it retains local `_profile`, `_completed`, and error state and checks `mounted` before UI mutations.

Step 3 - Service / Repository Layer

`RestoreSession.call()` invokes `AuthRepository.restoreSession()`. In live mode, `FirebaseAuthRepository.restoreSession()` waits for `FirebaseAuth.instance.authStateChanges().first`. If a user exists it calls `user.getIdToken()` and then authenticated `GET /api/auth/me/`.

Step 4 - Backend / API

HTTP: `GET /api/auth/me/`, header `Authorization: Bearer <Firebase ID token>`. DRF runs `FirebaseAuthentication.authenticate()`, which calls Firebase Admin token verification with `check_revoked=True`, finds `User(firebase_uid=decoded['uid'], is_active=True)`, and rejects a non-admin without a club. `accounts.views.MeView.get()` serializes the local user.

Step 5 - Database

Table `accounts_user`: query filter `firebase_uid=<decoded uid>` and active status. Primary key `id`; `club_id` is checked for non-admin. This is read-only.

Step 6 - Backend Response

Success: HTTP 200 JSON from `UserSerializer` containing local identity/role/club profile fields. Missing/invalid/revoked token yields authentication error; missing/inactive/local-unscoped user yields 401/403. Other network/server failures propagate as `AuthException`.

Step 7 - Model Conversion

JSON map -> `UserProfile.fromJson()` in `domain/entities/user_profile.dart` -> typed Dart profile and role enum.

Step 8 - State Update

On a non-null profile, `_restore()` also starts `registerDeviceProvider` without blocking and calls `attendanceSyncServiceProvider?.start()`. It assigns `_profile` and marks completion with `setState`. No profile means logged-out completion. A 401/403 path signs Firebase out inside the repository; a non-auth failure becomes the screen error.

Step 9 - UI Update

`build()` returns `HomeScreen(profile: _profile!)` on success, `LoginScreen` when null, or `_RestoreErrorScreen` with retry/continue behavior on a recoverable error.

Call chain: `initState()` -> `_restore()` -> `restoreSessionProvider` -> `RestoreSession.call()` -> `FirebaseAuthRepository.restoreSession()` -> `authStateChanges().first` -> `getIdToken()` -> `GET /api/auth/me/` -> `FirebaseAuthentication.authenticate()` -> `MeView.get()` -> `UserSerializer` -> `UserProfile.fromJson()` -> `setState()` -> `HomeScreen`.

Variable Trace: Firebase UID and role

Firebase session user -> ID token -> decoded backend `uid` -> `accounts_user.firebase_uid` lookup -> `User.role` -> serialized role string -> `UserProfile.role` -> `HomeScreen` switch.

Failure path: no Firebase user/token -> null -> login; 401/403 -> Firebase sign-out -> auth error/null path; connection/5xx -> `AuthException` -> restore error screen, allowing retry rather than silently authorizing cached data.

Workflow 3: Super Admin Club / Coordinator Provisioning

Trigger

No public registration exists. An authenticated Super Admin submits the protected Club or Coordinator API/admin form.

Step 1 - UI

Super Admin enters Club details and selects School or Independent, then selects that valid Club while creating its one Coordinator.

Step 2 - State / Controller Layer

There is no Flutter controller. `AdminClubSerializer` and `AdminCoordinatorCreateSerializer` validate protected input, including active Club existence and coordinator uniqueness.

Step 3 - Service / Repository Layer

The Club endpoint saves the existing affiliation fields. The Coordinator endpoint calls `provision_club_coordinator`, which fixes the role and selected Club server-side.

Step 4 - Backend / API

`POST /api/admin/clubs/` and `POST /api/admin/coordinators/` require `IsAdmin`. Unauthorized roles receive 403; invalid or duplicate Club selection receives 400.

Step 5 - Database

Insert into `accounts_club` and `accounts_user` with role `COORDINATOR`, the exact selected `club_id`, and a hashed Django portal password. A conditional database constraint permits only one Coordinator per Club.

Step 6 - Backend Response

Success returns 201 and a one-time portal credential. Invalid selection or authorization fails without partial relational state.

Step 7 - Model Conversion

No Dart model. DRF serializer data becomes Django model instances.

Step 8 - State Update

The created Club/Coordinator is immediately visible to Super Admin and can be activated/deactivated through protected lifecycle controls.

Step 9 - UI Update

The protected console/admin surface refreshes; the public signup page remains informational and non-mutating.

Call chain: Super Admin request -> protected serializer/view -> Club save -> `provision_club_coordinator()` -> role/Club validation -> Coordinator save -> response.

Failure path: non-admin -> 403; invalid/inactive Club or duplicate Coordinator -> 400; transaction error -> no partial Coordinator row.

> Mobile/public self-registration is not implemented. Normal Club accounts are provisioned by the authenticated Club Coordinator.

Workflow 4: Login

Trigger

The `ElevatedButton.onPressed` in `presentation/screens/login_screen.dart` near line 147 invokes `_handleSignIn()` unless `LoginState.isLoading` is true.

Step 1 - UI

`_handleSignIn()` reads `_emailController.text.trim()` and `_passwordController.text`, then calls `ref.read(loginControllerProvider.notifier).signIn(email: ..., password: ...)`. The button disables and shows progress from watched controller state.

Step 2 - State / Controller Layer

`LoginController.signIn()` in `presentation/providers/auth_controllers.dart` sets `isLoading=true` and clears the previous error. It calls `ref.read(signInProvider)(email: email, password: password)`. Success stores no global role itself; it returns `UserProfile`. Failure stores a user-facing error in `LoginState` and returns null. This layer isolates async UI state from authentication mechanics.

Step 3 - Service / Repository Layer

`SignIn.call()` calls `FirebaseAuthRepository.signInAndFetchProfile(email,password)`. The repository calls `FirebaseAuth.instance.signInWithEmailAndPassword`, obtains the current user ID token, calls `/api/auth/me/`, and converts the local profile. Firebase exceptions are mapped by `_friendlyAuthMessage()`.

Step 4 - Backend / API

Operation 1: Firebase email/password sign-in. Operation 2: `GET /api/auth/me/` with Bearer ID token. `FirebaseAuthentication` performs token/user/club checks; `MeView.get()` returns `UserSerializer(request.user).data`.

Step 5 - Database

Read `accounts_user` by `firebase_uid`; verify `is_active` and required `club_id`. No password is read from Django for mobile login.

Step 6 - Backend Response

HTTP 200 user profile on success. Firebase invalid credentials, token failure, network failure, missing local user, inactive user, or forbidden club state return exceptions. The repository also signs the Firebase user back out if application-profile authorization fails after Firebase login.

Step 7 - Model Conversion

`jsonDecode(response.body)` -> `Map<String,dynamic>` -> `UserProfile.fromJson()`.

Step 8 - State Update

`LoginController` sets loading false. `_handleSignIn()` receives the profile, starts device registration and attendance sync, then checks `mounted`.

Step 9 - UI Update

`Navigator.of(context).pushReplacement(MaterialPageRoute(builder: (_) => HomeScreen(profile: profile)))`. On failure, the watched controller error is rendered on the login screen; the button re-enables.

Call chain: login button -> `_handleSignIn()` -> `LoginController.signIn()` -> `SignIn.call()` -> `FirebaseAuthRepository.signInAndFetchProfile()` -> Firebase `signInWithEmailAndPassword()` -> `getIdToken()` -> `/api/auth/me/` -> `UserProfile.fromJson()` -> `HomeScreen`.

Variable Trace: Email/password

`_emailController.text.trim()` + `_passwordController.text` -> named parameters of `LoginController.signIn` -> `SignIn` -> `signInAndFetchProfile` -> Firebase credential call. The password is not sent to Django or stored in a Dart model.

Failure path: no explicit empty-field screen validation -> Firebase rejects -> mapped `AuthException` -> controller `error` -> login error UI. This is a client UX limitation; the backend authorization boundary remains intact.

Workflow 5: Logout

Trigger

Portal app-bar/profile sign-out actions call role-screen `_signOut(...)` or `HomeScreen._signOut()`; examples are `home_screen.dart`, `player_dashboard_screen.dart`, and `guardian_dashboard_screen.dart`.

Step 1 - UI

The icon/button `onPressed` invokes the screen's async `_signOut` method.

Step 2 - State / Controller Layer

The method first awaits `ref.read(unregisterDeviceProvider)()` and then awaits `ref.read(signOutProvider)()`. Player/Guardian paths also clear the in-memory privacy-unlock store before leaving the portal.

Step 3 - Service / Repository Layer

`UnregisterDevice.call()` delegates to `ApiDeviceRepository.unregisterCurrentDevice()`: it obtains the current FCM token with a five-second timeout and asks the authenticated API to delete that token. The repository catches/logs token or HTTP failures so cleanup can never block logout. `SignOut.call()` then delegates to `FirebaseAuthRepository.signOut()`, which signs out of Firebase and clears that owner's durable GET cache.

Step 4 - Backend / API

Before Firebase sign-out, Flutter sends `DELETE /api/devices/` with JSON `{token}` and the current Bearer token. `DeviceRegisterView.delete()` scopes deletion to `user=request.user`; it cannot remove another account's token association. No separate Django session-logout endpoint is used, and server-side Firebase token revocation is not part of ordinary local logout.

Step 5 - Database

On successful unregister, the matching current-user row in `academy_devicetoken` is deleted. No training, player, attendance, assessment, or other academy domain row is changed. If token lookup or transport fails, the row may remain until later cleanup, but sign-out still proceeds.

Step 6 - Backend Response

The unregister endpoint returns HTTP 204 whether or not the scoped token row existed. Flutter accepts 200/204 and swallows/logs unregister failures; Firebase sign-out then returns `Future<void>`.

Step 7 - Model Conversion

None.

Step 8 - State Update

Firebase local auth state becomes signed out; guardian unlock tokens are cleared from the memory store. Screen awaits the operation and checks `context.mounted` where used.

Step 9 - UI Update

`Navigator` removes/replaces the authenticated route with `LoginScreen`, so back navigation does not reopen the portal.

Call chain: sign-out button -> `_signOut()` -> `unregisterDeviceProvider` -> `UnregisterDevice.call()` -> `ApiDeviceRepository.unregisterCurrentDevice()` -> FCM token -> `DELETE /api/devices/` -> scoped `DeviceToken.delete()` -> `signOutProvider` -> `SignOut.call()` -> Firebase sign-out/current-owner cache clear -> clear unlock store -> Navigator to `LoginScreen`.

Failure path: token absence, timeout, or unregister HTTP failure is logged and does not prevent Firebase sign-out. A Firebase sign-out exception can still interrupt navigation; the exact screen methods do not all expose a rich recovery message. Only the current user's matching device-token row is eligible for deletion.

Workflow 6: Role Checking and Authorization

Trigger

Every authenticated API request triggers DRF authentication; every endpoint method then performs its role/object rules. `HomeScreen.build()` also uses `profile.role` for UX routing.

Step 1 - UI

Flutter hides/chooses role interfaces using `UserProfile.role`, but this is not the trusted enforcement. A modified client can still construct requests.

Step 2 - State / Controller Layer

`UserProfile` carries the serialized role. Controllers do not grant permission; they only initiate calls and surface errors.

Step 3 - Service / Repository Layer

API repositories attach only the current Firebase ID token, not a claimed role/club. This deliberately leaves authority to Django.

Step 4 - Backend / API

`FirebaseAuthentication.authenticate()` maps token UID to `accounts.User`. Views compare `request.user.role` to `Roles.*`, filter/query by `request.user.club_id`, derive player from `request.user` where appropriate, and call guardian-link/unlock helpers. `accounts/permissions.py` supplies reusable role permissions for admin endpoints.

Step 5 - Database

`accounts_user.role` and `club_id`, `accounts_guardianlink`, target object foreign keys, and `academy_playerprivacypin`/signed token state participate. There is no RLS query because Django is the only database client.

Step 6 - Backend Response

Allowed requests return serialized data; invalid identity returns 401; authenticated but disallowed role/object/club returns 403 (or a scoped 404 in selected cases). Serializer errors return 400; PIN lock can return 423.

Step 7 - Model Conversion

Successful role is parsed into the Dart role enum; errors become repository exceptions rather than domain objects.

Step 8 - State Update

Controller/provider becomes data or error. No client-side state can convert a 403 into authorized data.

Step 9 - UI Update

Home routing displays the role portal; feature widgets show error states when the backend rejects access.

Call chain: Firebase user -> ID token -> API repository header -> `FirebaseAuthentication.authenticate()` -> local `User` -> view role check -> club/object/link/unlock check -> ORM query/response.

Defense answer - player pretending to be coach: Changing a Flutter enum or payload does not change `request.user.role`. Django obtains UID from a verified token, loads `accounts_user.role`, and coach-only endpoints such as `PlayerAssessmentView.put` and `SessionAttendanceView.post` reject any non-coach.

Workflow 7: Navigation After Login

Trigger

Successful `_handleSignIn()` pushes `HomeScreen(profile: profile)`; restored sessions directly build the same destination.

Step 1 - UI

`HomeScreen.build()` switches on `profile.role`.

Step 2 - State / Controller Layer

The typed `UserProfile` came from server JSON. No additional controller is used for routing.

Step 3 - Service / Repository Layer

No new service call is needed to choose the initial portal; portal children then watch their own providers.

Step 4 - Backend / API

Role came from `/api/auth/me/`, not from a route parameter.

Step 5 - Database

Role came from `accounts_user.role` during the preceding profile read.

Step 6 - Backend Response

The response's role string is the routing input.

Step 7 - Model Conversion

`UserProfile.fromJson()` converts the wire role to the domain role enum.

Step 8 - State Update

No route-state framework; `HomeScreen` returns a different widget.

Step 9 - UI Update

`COACH` -> `CoachPortalScreen(profile)`; `PLAYER` -> `PlayerPortalScreen`; `GUARDIAN` -> `GuardianPortalScreen`; admin/coordinator/staff -> generic signed-in placeholder. Feature transitions use `Navigator.push/pushReplacement` with `MaterialPageRoute`; tab shells use `IndexedStack`.

Call chain: `/api/auth/me/` role -> `UserProfile.fromJson()` -> `HomeScreen.build()` switch -> role portal -> portal tab `IndexedStack`.

Failure path: unexpected/non-mobile role does not get coach capabilities; it receives a generic screen and can sign out.

Workflow 8: Player Profile Retrieval

Trigger

`PlayerDashboardScreen.build()` watches `myProfileProvider` after a player passes the privacy setup gate. Coach roster and guardian detail use separate providers but converge on `ApiPlayerRepository` and `Player.fromJson()`.

Step 1 - UI

Player dashboard consumes an `AsyncValue<Player>` and renders loading/error/data. Guardian unlocked content watches `selectedChildDetailsProvider(playerId)`; coach squad/profile uses `squadProvider`/selected `Player`.

Step 2 - State / Controller Layer

`myProfileProvider` calls `ref.watch(getMyProfileProvider)()`. It exists so widgets depend on typed async state rather than transport code.

Step 3 - Service / Repository Layer

`GetMyProfile.call()` calls `ApiPlayerRepository.fetchMyProfile()`, which delegates `_get('/api/players/me/')` to the shared `AuthenticatedApiClient`. The client resolves the current Firebase UID/token, applies the common timeout/error policy, and performs the GET. Guardian detail calls `fetchPlayerDetails(playerId, unlockToken)` and adds `X-Player-Unlock`; that protected response is deliberately excluded from durable caching.

Step 4 - Backend / API

Player: `GET /api/players/me/` -> `MyProfileView.get()` -> role must be `PLAYER` -> get `PlayerProfile(user=request.user)`. Guardian: `GET /api/players/{id}/profile/` -> `PlayerDetailView.get()` -> `_guardian_may_read` + `_require_unlock_when_pin_exists`. Coach roster: `GET /api/players/` -> `SquadListView.get()` and club filter.

Step 5 - Database

Read `academy_playerprofile` joined to `accounts_user`; nested fields include current ratings, eligibility, position, notes, and `photo_path`. Primary profile relation is one-to-one with player user. Storage helper may turn `photo_path` into a signed URL in serialization.

Step 6 - Backend Response

`PlayerSerializer` returns a player JSON object/list. Missing profile returns 404; wrong role/object returns 403; guardian may receive an unlock-related error.

Step 7 - Model Conversion

JSON -> `Player.fromJson()` -> nested `PlayerRatings.fromJson()`, `AgeTierInfo.fromWire()`, `PlayerPositionInfo.fromWire()`, and `EligibilityStatusLabel.fromWire()`.

Step 8 - State Update

Riverpod resolves the `FutureProvider` to `AsyncData<Player>`; exception becomes `AsyncError`. `autoDispose` releases state when no longer watched.

Step 9 - UI Update

`PlayerDashboardScreen`/guardian content uses cards, rating/eligibility widgets, and image fallback. Riverpod rebuilds consumers when the async value changes.

Call chain: `PlayerDashboardScreen` -> `myProfileProvider` -> `GetMyProfile.call()` -> `ApiPlayerRepository.fetchMyProfile()` -> `_get('/api/players/me/')` -> `MyProfileView.get()` -> `PlayerSerializer` -> `Player.fromJson()` -> `AsyncData` -> dashboard.

Object: Player

Created from `PlayerSerializer` JSON by `Player.fromJson`; stored transiently in Riverpod async state; consumed by dashboards, player cards, assessment/position screens; displays identity, tier, position, 12 ratings, eligibility, signed photo URL, and coach notes.

Workflow 9: Player Profile Update (Position)

The repository has no general player self-profile editor. The verified coach-controlled profile update is position assignment; assessment is Workflow 15.

Trigger

The coach opens a player profile/position picker and selects a `PlayerPosition`; the calling screen invokes the position controller's save action (provider in `presentation/providers/player_position_controller.dart`).

Step 1 - UI

`position_picker_sheet.dart` returns the chosen enum to the caller. The player ID comes from the selected `Player`; the wire value comes from `PlayerPosition.wire`.

Step 2 - State / Controller Layer

`PlayerPositionController` enters async loading and invokes `savePlayerPositionProvider(playerId, position)`. Success invalidates squad/player-derived state; failure stores `AsyncError`.

Step 3 - Service / Repository Layer

`SavePlayerPosition.call()` delegates to `ApiPlayerRepository.savePosition(playerId, position)`, which sends JSON `{'position': position.wire}` after obtaining an ID token.

Step 4 - Backend / API

`PUT /api/players/{playerId}/position/` with Bearer token and JSON. `PlayerPositionView.put()` requires `COACH`, loads `PlayerProfile`, verifies equal `club_id`, validates `PlayerPositionSerializer`, saves, and audits.

Step 5 - Database

Update `academy_playerprofile.position` for the row whose one-to-one user ID equals `playerId`; add `academy_auditlog` action `position.changed` with actor FK.

Step 6 - Backend Response

HTTP 200 `PlayerSerializer(profile).data`; 403 for non-coach/cross-club, 404 missing player, 400 invalid position. Repository maps non-200 to `PlayerRepositoryException`.

Step 7 - Model Conversion

Response JSON -> `Player.fromJson()` with `PlayerPositionInfo.fromWire()`.

Step 8 - State Update

Controller clears loading, returns updated `Player`, and invalidates roster state.

Step 9 - UI Update

The sheet/screen closes or updates, and rebuilt roster/profile cards display the new position.

Call chain: position selection -> `PlayerPositionController` -> `SavePlayerPosition.call()` -> `ApiPlayerRepository.savePosition()` -> `PUT /api/players/{id}/position/` -> `PlayerPositionView.put()` -> `PlayerPositionSerializer.save()` -> `PlayerSerializer` -> `Player.fromJson()` -> provider invalidation -> position label.

Failure path: no selection/cancel -> no request; invalid/cross-club/non-coach -> backend error -> repository/controller error -> UI remains unchanged.

Workflow 10: Creating a Scouting Report

`NOT IMPLEMENTED IN CURRENT REPOSITORY`

Trigger

No scouting-report button, form, widget, event handler, or `SCOUT` role exists.

Step 1 - UI

No scouting-report screen/file exists.

Step 2 - State / Controller Layer

No scouting controller/provider/use case exists.

Step 3 - Service / Repository Layer

No scouting repository/service exists.

Step 4 - Backend / API

No scouting endpoint or method exists.

Step 5 - Database

No scouting table/model/migration exists.

Step 6 - Backend Response

Not applicable.

Step 7 - Model Conversion

No scouting Dart model exists.

Step 8 - State Update

Not applicable.

Step 9 - UI Update

Not applicable. The coach dispute flow must not be presented as scouting.

Workflow 11: Retrieving Scouting Reports

`NOT IMPLEMENTED IN CURRENT REPOSITORY`

Trigger

No report-list trigger exists.

Step 1 - UI

No scouting list/card exists.

Step 2 - State / Controller Layer

No scouting read provider exists.

Step 3 - Service / Repository Layer

No retrieval method exists.

Step 4 - Backend / API

No scouting GET route exists.

Step 5 - Database

No scouting records exist.

Step 6 - Backend Response

Not applicable.

Step 7 - Model Conversion

Not applicable.

Step 8 - State Update

Not applicable.

Step 9 - UI Update

Not applicable.

Workflow 12: Editing a Scouting Report

`NOT IMPLEMENTED IN CURRENT REPOSITORY`

Trigger

No scouting edit action exists.

Step 1 - UI

No edit form.

Step 2 - State / Controller Layer

No edit controller.

Step 3 - Service / Repository Layer

No update repository method.

Step 4 - Backend / API

No PUT/PATCH scouting endpoint.

Step 5 - Database

No scouting table to update.

Step 6 - Backend Response

Not applicable.

Step 7 - Model Conversion

Not applicable.

Step 8 - State Update

Not applicable.

Step 9 - UI Update

Not applicable.

Workflow 13: Training Session Creation

Trigger

The add/schedule button in `presentation/screens/training_schedule_screen.dart` calls `_openScheduleForm()`, which pushes `ScheduleSessionScreen`. Its save `ElevatedButton.onPressed` near line 324 calls `_submit()`.

Step 1 - UI

`ScheduleSessionScreen` holds title/location controllers and selected `_date`, `_startTime`, `_endTime`, `_selectedTiers`, `_focus`. `_submit()` rejects empty fields, missing tiers, and invalid date conditions, then constructs an immutable `TrainingSession` draft. The server remains authoritative for the paired time-window invariant.

Step 2 - State / Controller Layer

`ScheduleSessionController.submit(draft)` in `training_schedule_providers.dart` calls `_run`, sets `AsyncLoading`, awaits `scheduleTrainingSessionProvider(draft)`, and stores `AsyncData` or `AsyncError`. It prevents duplicate save through `isLoading` UI disabling.

Step 3 - Service / Repository Layer

`ScheduleTrainingSession.call(draft)` calls `TrainingRepository.createSession`. `ApiTrainingRepository.createSession()` converts `draft.toJson()` and delegates the authenticated POST to `AuthenticatedApiClient`.

Step 4 - Backend / API

`POST /api/training-sessions/`; headers Bearer token + JSON content type. Payload: `id`, `title`, `ageTiers`, ISO date-only `date`, display-string `startTime`, `endTime`, `location`, uppercase `focus`, and `attendeeCount`. `TrainingSessionListCreateView.post()` requires coach; `TrainingSessionSerializer.validate()` calls `TrainingSession.validate_time_window()` so start/end must be supplied together, match the 12-hour wire format, and end later on the same day. The model repeats this through `clean()/save()` for non-API writes. The view assigns `created_by=request.user`, `club=request.user.club`.

Step 5 - Database

Insert `academy_trainingsession`: generated `id` PK, title/date/start/end/location/focus, JSON age tiers, `created_by_id` FK to `accounts_user`, `club_id` FK to `accounts_club`, timestamps. Insert an `academy_auditlog` event. On commit, notification code reads related club users/device tokens.

Step 6 - Backend Response

HTTP 201 with `TrainingSessionSerializer(session).data`. Invalid form/payload -> 400; non-coach/cross-auth -> 403/401; DB failure -> 5xx/rollback. Flutter accepts 200/201 and otherwise throws `TrainingRepositoryException`.

Step 7 - Model Conversion

Response JSON -> `TrainingSession.fromJson()`: string ID, parsed date, tier enum set, focus enum, attendee count.

Step 8 - State Update

Controller resolves; the caller/provider invalidates session data (through its workflow wiring) and `_submit()` receives success. Error is read from controller state.

Step 9 - UI Update

Success `Navigator.pop(true)` returns to schedule and refreshed providers rebuild session lists/cards. Failure shows a `SnackBar`; the form remains.

Call chain: add button -> `_openScheduleForm()` -> `ScheduleSessionScreen` -> save button -> `_submit()` -> `ScheduleSessionController.submit()` -> `ScheduleTrainingSession.call()` -> `ApiTrainingRepository.createSession()` -> `TrainingSession.toJson()` -> `POST /api/training-sessions/` -> `TrainingSessionListCreateView.post()` -> serializer/model/audit/on-commit FCM -> `TrainingSession.fromJson()` -> pop/refresh.

Variable Trace: Training ID

New draft uses a client placeholder/empty ID -> server ignores/generates model PK -> serializer returns `id` -> `TrainingSession.fromJson` stores it as String -> edit/cancel/log-attendance routes use `session.id`.

Failure path: UI validation prevents an incomplete request; missing token throws "Not signed in"; malformed/unpaired/reversed times return field-specific 400 errors; a transport failure maps to the shared reach-server error; another non-success status remains an HTTP error; controller `AsyncError` -> SnackBar.

Workflow 14: Training Attendance Recording

Trigger

Coach taps a session card in `training_schedule_screen.dart`; `_logAttendance(session)` pushes `LogAttendanceScreen`. In that screen, `_FinalizeBar` button near line 915 calls `_finalize(roster)` when the session window is open and at least one mark exists.

Step 1 - UI

`_marks: Map<String, AttendanceStatus?>`, effort values, and note fields hold inputs. `_finalize()` checks the two-day attendance window, handles unmarked-player confirmation, and builds `List<Attendance>`. Effort/note are included only for `PRESENT`; non-present records clear them.

Step 2 - State / Controller Layer

`AttendanceLogController.save(session.id, records)` sets `AsyncLoading`, awaits `logSessionAttendanceProvider(sessionId,records)`, invalidates `sessionAttendanceProvider(sessionId)` on success, and stores `AsyncError` on failure. This separates transient field state from mutation state.

Step 3 - Service / Repository Layer

`LogSessionAttendance.call()` invokes the injected `AttendanceRepository`. Live composition is `OfflineFirstAttendanceRepository`, which first calls `ApiAttendanceRepository.saveSessionAttendance()`. The API adapter creates `{'records': records.map(toJson)}`. Only `AttendanceNetworkException` causes the decorator to require `_ownerUid`, enqueue JSON, and return optimistic records.

Step 4 - Backend / API

`POST /api/attendance/session/{sessionId}/`, Bearer + JSON. `SessionAttendanceView.post()` requires `COACH`, same-club session, and days-since 0-2. `SessionAttendanceRecordSerializer(many=True)` validates each row. It verifies every `playerId` belongs to the coach's club.

Step 5 - Database

Within `transaction.atomic()`, for each validated row: `Attendance.objects.update_or_create(player_id, session, defaults={status, effort, note, recorded_by})`. Table `academy_attendance`; PK `id`; FKs `player_id`, nullable `session_id`, `recorded_by_id`; unique pair `(player,session)`. Existing rows for the session whose player IDs were omitted are deleted. Offline branch instead inserts `outbox_attendance(owner_uid,session_id,records_json,created_at,retry_count,last_error)` on device.

Step 6 - Backend Response

HTTP 200 list from `AttendanceSerializer`. Possible 400: window/record validation; 401/403: identity/role/club; network: typed network exception; 5xx: generic repository error. Offline queueing occurs only on network exception, not HTTP errors.

Step 7 - Model Conversion

JSON list -> `_decodeRecords()` -> `Attendance.fromJson()` for each row. Offline optimistic objects are the same `Attendance` instances created by the UI; queued JSON later becomes `Attendance.fromJson()` during replay.

Step 8 - State Update

Online: controller becomes data and invalidates session attendance. Offline: decorator returns records so controller treats it as accepted-for-sync; outbox persists pending state. `AttendanceSyncService.start()` listens/initiates replay; it processes owner rows oldest first, deletes success, increments retry/error and stops on failure.

Step 9 - UI Update

Success/queued acceptance pops `LogAttendanceScreen(true)` and shows caller feedback. Refetched session/player attendance cards display saved status/effort/note after server sync. Failure keeps the screen and shows controller error through a SnackBar.

Call chain: session card -> `_logAttendance()` -> `LogAttendanceScreen` -> finalize button -> `_finalize()` -> `AttendanceLogController.save()` -> `LogSessionAttendance.call()` -> `OfflineFirstAttendanceRepository.saveSessionAttendance()` -> `ApiAttendanceRepository.saveSessionAttendance()` -> POST -> `SessionAttendanceView.post()` -> atomic upsert/prune -> `AttendanceSerializer` -> `Attendance.fromJson()` -> invalidate/pop.

Offline call chain: API transport failure -> `AttendanceNetworkException` -> `AttendanceOutbox.enqueue()` -> local `outbox_attendance` -> `AttendanceSyncService` -> live repository retry -> Django endpoint -> success -> outbox delete.

Variable Trace: Attendance status

Selector enum `AttendanceStatus` -> `_marks[player.id]` -> `Attendance(status: ...)` -> `Attendance.toJson()['status']` uppercase -> request `records[]` -> serializer choice -> `academy_attendance.status` -> `AttendanceSerializer.status` -> `AttendanceStatusWire.fromWire()` -> chip/summary UI.

Failure path: unmarked/closed window -> SnackBar/dialog, no API; missing token -> repository error, no queue; connection failure -> queue if owner exists, otherwise "Not signed in"; backend 400/403/500 -> no queue and controller error; replay failure -> retry metadata retained.

Workflow 15: Coach Evaluation / Assessment

Trigger

Coach opens `EditPerformanceDataScreen` for a player and presses its save `ElevatedButton.onPressed` near line 246, invoking `_save()`.

Step 1 - UI

The screen's rating values/controllers and coach-notes controller contain the input. `_save()` constructs `PlayerRatings` for the six outfield and six goalkeeper attributes and reads trimmed notes, then calls the controller with the selected `Player`.

Step 2 - State / Controller Layer

`EditPerformanceController.submit(player, ratings, coachNotes)` sets `AsyncLoading`, calls `savePlayerAssessmentProvider`, stores `AsyncData` and invalidates squad state on success, or `AsyncError` and returns null on failure.

Step 3 - Service / Repository Layer

`SavePlayerAssessment.call(player.id, ratings, coachNotes: ...)` -> `ApiPlayerRepository.saveAssessment()`. It JSON-encodes `{'ratings': ratings.toJson(), 'coachNotes': coachNotes}` and delegates the authenticated PUT to `AuthenticatedApiClient`.

Step 4 - Backend / API

`PUT /api/players/{playerId}/assessment/`. `PlayerAssessmentView.put()` requires `COACH`, loads the profile, compares player/coach `club_id`, runs `AssessmentSerializer(profile,data,partial=True)`, saves, audits `assessment.saved`, and schedules `notify_assessment_saved(profile)` on commit.

Step 5 - Database

Update the existing `academy_playerprofile` row: `pace`, `shooting`, `passing`, `dribbling`, `defending`, `physical`, `diving`, `handling`, `kicking`, `reflexes`, `speed`, `positioning`, and `coach_notes`. Add `academy_auditlog`. No assessment-history row is created; the current values overwrite.

Step 6 - Backend Response

HTTP 200 `PlayerSerializer(profile).data`. Serializer rejects out-of-range ratings (0-99 contract) or malformed data. Wrong role/club yields 403; missing player 404; transport/non-200 becomes `PlayerRepositoryException`.

Step 7 - Model Conversion

JSON -> `Player.fromJson()` -> nested `PlayerRatings.fromJson()` and related enums.

Step 8 - State Update

Controller leaves loading, returns updated `Player`, and invalidates the squad provider so all consumers can reload authoritative data.

Step 9 - UI Update

`_save()` receives non-null `saved`, calls `Navigator.pop(saved)`, and previous player/card screens update from the return/refetch. Failure reads controller error and displays a SnackBar.

Call chain: save evaluation button -> `_save()` -> `EditPerformanceController.submit()` -> `SavePlayerAssessment.call()` -> `ApiPlayerRepository.saveAssessment()` -> `PUT /api/players/{id}/assessment/` -> `PlayerAssessmentView.put()` -> `AssessmentSerializer.save()` -> profile/audit/on-commit push -> `PlayerSerializer` -> `Player.fromJson()` -> provider invalidation/pop.

Variable Trace: Performance score

Rating UI integer -> `PlayerRatings.<field>` -> `ratings.toJson()` -> nested `ratings` request -> `AssessmentSerializer` validated integer 0-99 -> `PlayerProfile.<field>` -> serializer nested ratings -> `PlayerRatings.fromJson()` -> `Player.overall` averages applicable six -> card number/radar UI.

Failure path: malformed/range input should be stopped by UI/serializer; network/non-200 -> controller `AsyncError` -> SnackBar; database rollback prevents response and on-commit push. There is no old assessment record to recover except external backup/audit metadata.

Workflow 16: Performance Data Retrieval

Trigger

Coach/player/guardian screens watch a player provider; for example `PlayerDashboardScreen` watches `myProfileProvider`, and coach roster screens watch `squadProvider`.

Step 1 - UI

A `ConsumerWidget` reads an `AsyncValue<Player>` or `AsyncValue<List<Player>>`; current ratings and coach notes are rendered on player cards/profile/radar widgets.

Step 2 - State / Controller Layer

`myProfileProvider` -> `getMyProfileProvider`; `squadProvider` -> `getSquadProvider`; guardian full detail -> `selectedChildDetailsProvider` -> `getPlayerDetailsProvider`. These `FutureProviders` hold loading/data/error state.

Step 3 - Service / Repository Layer

The corresponding use cases call `ApiPlayerRepository.fetchMyProfile()`, `fetchSquad()`, or `fetchPlayerDetails()`. The repository adds Firebase auth and, for guardians, an unlock header.

Step 4 - Backend / API

`GET /api/players/me/`, `GET /api/players/`, or `GET /api/players/{id}/profile/`. `MyProfileView`, `SquadListView`, or `PlayerDetailView` applies role/club/link/PIN rules and returns `PlayerSerializer`.

Step 5 - Database

Read the current row in `academy_playerprofile` and related `accounts_user`; columns include the 12 ratings and `coach_notes`. No historical assessment query exists.

Step 6 - Backend Response

One player map or list of maps. Null/missing optional position/photo/notes are normalized by serializer/model defaults. 401/403/404/network errors become provider error states.

Step 7 - Model Conversion

JSON -> `Player.fromJson()` -> `PlayerRatings.fromJson()`; `Player.overall` deterministically averages the relevant six attributes based on position group.

Step 8 - State Update

Riverpod `FutureProvider` resolves to `AsyncData`; refetch occurs after assessment/position provider invalidation.

Step 9 - UI Update

Player card, attribute radar/stat widgets, overall value, eligibility badge, and coach-note text rebuild from the typed `Player`.

Call chain: player consumer -> player `FutureProvider` -> get-player use case -> `ApiPlayerRepository` GET -> Django player view -> `PlayerProfile`/`PlayerSerializer` -> `Player.fromJson()`/`PlayerRatings.fromJson()` -> `AsyncData` -> card/radar/text.

Failure path: auth/network/server exception -> `AsyncError` -> dashboard-state error/retry UI; a missing optional photo uses avatar fallback rather than failing the whole player.

Workflow 17: Performance Dashboard Generation

Trigger

Opening `presentation/screens/coach_progress_screen.dart` causes `CoachProgressScreen.build()` to watch `squadProgressProvider`.

Step 1 - UI

The `ConsumerWidget` renders loading, error, or a squad summary plus `_PlayerProgressCard` list.

Step 2 - State / Controller Layer

`squadProgressProvider` in `progress_providers.dart` calls `ref.watch(getSquadProgressProvider)()` and owns `AsyncValue<List<PlayerProgress>>`.

Step 3 - Service / Repository Layer

`GetSquadProgress.call()` -> `ApiProgressRepository.fetchSquadProgress()` -> shared `AuthenticatedApiClient` -> authenticated GET.

Step 4 - Backend / API

`GET /api/progress/squad/`. `SquadProgressView.get()` allows Coach/Super Admin. It filters profiles and attendance to `request.user.club_id` only for a Coach; the Super Admin keeps the all-club querysets. It uses ORM `Count` filters plus `Avg('effort')`.

Step 5 - Database

Read `academy_playerprofile` joined with `accounts_user`; aggregate `academy_attendance` grouped by `player_id`. Returned values: present/absent/excused counts and average effort; no row is inserted.

Step 6 - Backend Response

HTTP 200 list of `{id,name,position,ageTier,present,absent,excused,avgEffort}`. Players with no attendance receive zeros/null average. Non-coach/admin gets 403.

Step 7 - Model Conversion

JSON list -> `.map(PlayerProgress.fromJson)` -> typed immutable progress objects.

Step 8 - State Update

Provider resolves `AsyncData<List<PlayerProgress>>`; error becomes `AsyncError` and refresh can rerun the future.

Step 9 - UI Update

`_SquadSummary` and each `_PlayerProgressCard` calculate/display the returned deterministic metrics.

Call chain: Progress tab -> `CoachProgressScreen` -> `squadProgressProvider` -> `GetSquadProgress.call()` -> `ApiProgressRepository.fetchSquadProgress()` -> `SquadProgressView.get()` -> ORM Count/Avg -> `PlayerProgress.fromJson()` -> cards.

Failure path: network/non-200 -> error UI. Role branching is explicit: Coach is club-scoped; Super Admin receives genuine all-club progress rather than filtering on the Admin's normally-null club.

Workflow 18: Academic Eligibility Retrieval

Trigger

Player/guardian taps the eligibility tile; `Navigator.push` opens `EligibilityHistoryScreen`, whose `build()` watches `eligibilityHistoryProvider(playerId)`.

Step 1 - UI

`EligibilityHistoryScreen` receives `playerId` and uses a `ConsumerWidget` to render loading/error/list of `_ChangeCard` records.

Step 2 - State / Controller Layer

The family `FutureProvider` invokes `getEligibilityHistoryProvider(playerId)`; guardian token state is injected through the repository composition.

Step 3 - Service / Repository Layer

`GetEligibilityHistory.call()` -> `ApiEligibilityHistoryRepository.fetchHistoryForPlayer(playerId)` -> ID token + optional `X-Player-Unlock` -> HTTP GET.

Step 4 - Backend / API

`GET /api/players/{playerId}/eligibility-history/`. `EligibilityHistoryView.get()` calls `_may_read_eligibility`, verifies the player, applies PIN unlock when configured, and serializes ordered history.

Step 5 - Database

Read `academy_eligibilityhistory` filtered `player_id`, joined to optional `changed_by_id`; player FK references `accounts_user`. Current status itself is also present on `academy_playerprofile.eligibility`.

Step 6 - Backend Response

HTTP 200 history list; empty list is valid. Role/link/PIN failure -> 403/validation response; missing privileged target may be 404. Actor display is shaped for the viewer and may be null.

Step 7 - Model Conversion

Each JSON map -> `EligibilityChange.fromJson()` with old/new status and timestamp.

Step 8 - State Update

Riverpod family provider becomes data/error for that player ID.

Step 9 - UI Update

List cards show chronological eligibility transitions. Eligibility badge/current status elsewhere comes from the player profile.

Call chain: eligibility tile -> `Navigator.push(EligibilityHistoryScreen)` -> `eligibilityHistoryProvider(id)` -> `GetEligibilityHistory.call()` -> API repository -> GET -> `EligibilityHistoryView.get()` -> history queryset/serializer -> `EligibilityChange.fromJson()` -> list.

Failure path: guardian without valid in-memory unlock receives denial; provider renders error/retry. No grades are retrieved because no grade data exists.

Workflow 19: Academic Eligibility Update

Trigger

School staff submits the server-rendered form handled by `backend/portal/views.py:staff_eligibility(request)` near line 314.

Step 1 - UI

The Django portal page posts selected club player and new eligibility status using a CSRF-protected form. There is no Flutter eligibility-edit button.

Step 2 - State / Controller Layer

No Flutter controller. Django form validation checks the supplied choice/player; portal decorators/session middleware ensure authenticated staff role.

Step 3 - Service / Repository Layer

The view calls `portal.services.set_player_eligibility(staff=request.user, player_profile=..., new_status=...)`. `_assert_same_club`/service logic prevents cross-club updates, sets the actor marker, and saves the profile.

Step 4 - Backend / API

Session-authenticated portal POST, CSRF protected. It is not a public DRF PUT. Django role decorator and service enforce `SCHOOL_STAFF` and club scope.

Step 5 - Database

Update `academy_playerprofile.eligibility`. `signals.stash_previous_eligibility` reads the previous value before save. `fire_eligibility_changed` inserts `academy_eligibilityhistory(player_id,changed_by_id,old_status,new_status,changed_at)` plus `academy_auditlog` and schedules FCM after commit.

Step 6 - Backend Response

Valid update redirects/rerenders with a success message. Form, permission, or service validation rerenders/rejects; transaction/save failure prevents a normal success response.

Step 7 - Model Conversion

No immediate Dart conversion in the portal. Later mobile retrieval converts history with `EligibilityChange.fromJson()` and current status with `Player.fromJson()`.

Step 8 - State Update

Portal messages/session response update. Mobile providers see the new data on refresh/refetch; no real-time stream automatically patches the screen.

Step 9 - UI Update

Portal shows success/current value. Refreshed player/guardian mobile badge and history screen display the updated status/event.

Call chain: staff form submit -> `staff_eligibility()` -> form/role/club validation -> `set_player_eligibility()` -> `PlayerProfile.save()` -> `stash_previous_eligibility()` -> `fire_eligibility_changed()` -> history/audit + on-commit FCM -> redirect -> later mobile GET/refetch.

Variable Trace: Eligibility status

Form choice -> cleaned `new_status` -> service -> `PlayerProfile.eligibility` -> pre/post signal old/new -> `EligibilityHistory.old_status/new_status` -> serializer -> `EligibilityChange/Player.eligibility` -> badge/cards.

Failure path: unauthorized role/club -> forbidden; invalid choice/player -> form error; unchanged value -> no new change event; notification failure is designed not to invalidate the saved status.

Workflow 20: Notifications

Trigger

Three linked triggers exist: (a) successful login/restore calls `registerDeviceProvider`; (b) schedule/assessment/eligibility mutations call notification helpers, usually through `transaction.on_commit`; and (c) a bell tap, inbox-row tap, foreground FCM **View** action, opened push, or initial push enters the notification read/routing flow.

Step 1 - UI

Registration is initiated without blocking the login/bootstrap UI. `NotificationBell` watches the unread-count provider, displays a badge, and pushes `NotificationInboxScreen`. In live mode, a foreground message shows a SnackBar with a **View** action. That action, an opened/initial push, and an inbox-row tap use the same destination policy: `session_*` enters the role portal's schedule tab; `assessment_saved` enters the Player's own or Guardian's authorized child profile tab; `eligibility_changed` opens eligibility history for the Player or authorized child; unknown/unsupported events stay in a focused inbox. Player payload IDs are ignored in favor of the current profile, and a Guardian payload ID is only a hint that must match the linked-child list; protected destinations still enforce PIN/privacy gates.

Step 2 - State / Controller Layer

`RegisterDevice` is invoked after auth. `notificationsProvider`, `notificationUnreadCountProvider`, and `notificationActionsProvider` own inbox loading and read-state refresh. `notificationNavigationControllerProvider` re-fetches the server-authoritative current profile, resolves a role-appropriate `NotificationDestination`, suppresses duplicate active callbacks, and marks the represented inbox record read best-effort. Mutation controllers do not send notifications directly; backend business events do.

Step 3 - Service / Repository Layer

Flutter `ApiDeviceRepository` obtains `FirebaseMessaging.instance.getToken()`/platform data and POSTs through the `DeviceRepository` contract. `ApiNotificationRepository` uses `AuthenticatedApiClient` for list/count/mark-read operations and converts JSON to `AppNotification`. Backend `academy.notifications._send_to_users()` resolves active recipients, persists one `NotificationRecord` per recipient, and then performs best-effort Firebase Admin fanout; invalid tokens can be removed.

Step 4 - Backend / API

Registration: `POST /api/devices/` JSON `{token, platform}` with Bearer auth -> `DeviceRegisterView.post()`. Logout cleanup uses `DELETE /api/devices/` JSON `{token}` -> `DeviceRegisterView.delete()`, scoped to the current user. Inbox APIs are `GET /api/notifications/`, `GET /api/notifications/unread-count/`, `PATCH /api/notifications/{id}/read/`, and `POST /api/notifications/read-all/`; every query is scoped to `request.user`. Sending is triggered for session scheduled/updated/cancelled, assessment saved, and eligibility changed. For cancellation, the view snapshots recipients, deletes the session, and schedules `notify_session_cancelled(...)` with `transaction.on_commit`, so a failed delete cannot emit a false cancellation.

Step 5 - Database

`academy_devicetoken`: registration uses `update_or_create(token=..., defaults={user,platform})`; logout deletes only a row matching both the authenticated user and token. The token is unique and references `accounts_user`. `academy_notificationrecord` stores recipient, event type, title, body, neutral JSON data, nullable `read_at`, and `created_at`, indexed by user/read/time. Notification events create inbox rows before attempting FCM.

Step 6 - Backend Response

Device registration and unregister return HTTP 204. Inbox endpoints return the current user's newest records/count/read result. FCM failures are logged/swallowed after inbox persistence, so the business write and notification history survive unavailable Firebase; exact device delivery still depends on Firebase, Apple/Android configuration, OS permission, and device state.

Step 7 - Model Conversion

Inbox JSON is converted by `AppNotification.fromJson()` to a Dart object containing ID, type, title, body, data, read state, and timestamp. An inbox row or FCM data map becomes `NotificationOpenRequest`; current role plus event type becomes a `NotificationDestination`. IDs remain routing hints rather than authorization grants. Device registration still uses primitive token/platform strings.

Step 8 - State Update

Device-token and token-refresh registration keep delivery endpoints current. Foreground/opened handlers invalidate inbox/count providers; mark-one/mark-all actions also invalidate those providers after success. Opening a notification marks the exact ID, or the newest matching type/domain-ID row, read best-effort. An active-request key prevents duplicate navigation callbacks for the same delivery while its route operation is in progress.

Step 9 - UI Update

Foreground pushes display a SnackBar; the bell shows the inbox with loading, retry, pull-to-refresh, unread styling, and mark-read actions. Known opened/initial/foreground/inbox-row events enter the schedule, profile, or eligibility destination. Guardian child selection is resolved from the authenticated linked-child provider, and Player/Guardian privacy gates remain in force. Unknown events and profile-resolution failures fall back to the focused current-user inbox.

Call chains: login success -> `registerDeviceProvider` -> `RegisterDevice` -> `ApiDeviceRepository.registerCurrentDevice()` -> Firebase messaging token -> `POST /api/devices/` -> `DeviceRegisterView.post()` -> `DeviceToken.update_or_create()`.

Business mutation -> transaction commit -> `notify_*` helper -> `_send_to_users()` -> `NotificationRecord.bulk_create()` -> device-token queryset -> best-effort Firebase Admin FCM send.

Bell -> `NotificationInboxScreen` -> notification providers/repository -> current-user inbox endpoints -> `NotificationRecordSerializer` -> `AppNotification.fromJson()` -> badge/list/read-state UI.

Opened/initial/foreground **View**/inbox row -> `NotificationOpenRequest` -> `NotificationNavigationController.open()` -> re-resolve `/api/auth/me/` -> `resolveNotificationDestination()` -> best-effort mark-read + duplicate suppression -> `NotificationDestinationScreen` -> role portal initial tab/protected detail, or focused inbox fallback.

Failure path: token unavailable/request failure does not block login because registration is unawaited; invalid/stale FCM tokens can be cleaned. FCM/permission/device-delivery failure does not remove the persisted inbox record. Inbox network/server errors render retryable UI. A missing current profile or unknown event falls back to the inbox; an untrusted child ID cannot bypass linked-child lookup or privacy gates. Inbox/router behavior is covered by local mocked tests, while real Firebase/APNs transport and notification presentation still require a signed physical-device verification.

Workflow 21: File/Image Upload

Trigger

There are two UI entry points: a Coordinator portal file form posts to `portal.views.player_photo(request, player_id)`, and a Coach viewing a roster player taps **Update player photo** in `PlayerProfileScreen`, which invokes `_pickAndUploadPhoto()`. The protected DRF route is `POST /api/players/{id}/photo/` with the older `/api/admin/players/{id}/photo/` alias handled by the same `PlayerPhotoUploadView.post()`.

Step 1 - UI

The Coordinator selects an image in the server-rendered portal. In Flutter, the Coach uses `ImagePicker` to select a gallery image; the screen accepts JPEG/PNG/WebP content types, reads bytes, disables the action while uploading, and displays success or failure feedback.

Step 2 - State / Controller Layer

Flutter calls `PlayerPhotoController.submit()`, which invokes the `UploadPlayerPhoto` use case through `PlayerPhotoWriter`, stores loading/error state, and invalidates `squadProvider` after success. Django receives `request.FILES['photo']` through multipart parsing/form POST.

Step 3 - Service / Repository Layer

`ApiPlayerRepository.uploadPhoto()` delegates an authenticated multipart `photo` part to `AuthenticatedApiClient.postMultipart()`. The portal/API then calls `academy.storage.validate_photo_upload(upload)`, reads bytes, and invokes `upload_photo(user_id, content, content_type)`. Storage configuration comes from server environment only.

Step 4 - Backend / API

Portal POST is session/CSRF/Coordinator/club protected. `PlayerPhotoUploadView` permits a same-Club Coach or the Super Admin and rejects a cross-club Coach. Django then calls Supabase Storage REST using the server service credential. Flutter never receives that credential and never calls Storage directly.

Step 5 - Database

After storage success, update `academy_playerprofile.photo_path` with the private object path. `PlayerSerializer` later calls `signed_photo_url(photo_path)` to expose a time-limited URL.

Step 6 - Backend Response

Portal redirects/messages; API returns HTTP 200 serialized player. Missing file, invalid signature/MIME, over 25 MB, unconfigured Supabase credentials, or upload failure becomes a visible validation/controller error. Signed-read failure may return no URL so UI can use a fallback.

Step 7 - Model Conversion

API/profile JSON `photoUrl` -> `Player.fromJson()` `String? photoUrl`. The private service key and object bytes never become a Dart model.

Step 8 - State Update

Portal updates DB and redirects. Flutter replaces its local `_player` with the serialized response immediately and invalidates the squad for the next roster read.

Step 9 - UI Update

Portal feedback confirms the upload; Flutter shows a success SnackBar and the returned avatar immediately. Player avatars use `NetworkImage(url)` when nonempty; otherwise a default/initial avatar is rendered.

Call chains: Coach update action -> `_pickAndUploadPhoto()` -> `ImagePicker` -> `PlayerPhotoController.submit()` -> `UploadPlayerPhoto.call()` -> `ApiPlayerRepository.uploadPhoto()` -> `AuthenticatedApiClient.postMultipart()` -> `PlayerPhotoUploadView.post()` -> `validate_photo_upload()` -> Supabase REST -> `PlayerProfile.photo_path` -> `PlayerSerializer` -> `Player.fromJson()` -> local avatar/squad refresh.

Portal upload submit -> `player_photo()` -> `validate_photo_upload()` -> `upload_photo()` -> Supabase Storage REST -> `PlayerProfile.photo_path` save -> redirect -> later `PlayerSerializer` -> `signed_photo_url()` -> `Player.fromJson()` -> `NetworkImage`.

Failure path: picker cancellation -> no request; unsupported client type -> SnackBar; auth/cross-club/validation/storage error -> no `photo_path` update and visible form/API error; signed URL failure -> null photo URL/fallback. The upload path requires configured Supabase server credentials; implementation alone is not proof of a successful live storage upload.

Workflow 22: Search / Filtering

Trigger

Typing into the coach roster search/filter controls updates local filter state and causes the roster widget/provider selection to recompute `RosterFilter.apply`.

Step 1 - UI

Search text and selected tier/position are captured by text/control callbacks in the squad/coach UI.

Step 2 - State / Controller Layer

The roster filtering helper/provider receives the already loaded `List<Player>` plus criteria and returns a filtered list. State is local/Riverpod depending on the control; no mutation controller is needed.

Step 3 - Service / Repository Layer

Initial data came from `GetSquad/ApiPlayerRepository.fetchSquad()`. Changing search does not call the repository again.

Step 4 - Backend / API

Only initial `GET /api/players/` occurs. `SquadListView.get()` returns same-club players for coach. No `?search=` query is generated.

Step 5 - Database

Initial query reads `academy_playerprofile/accounts_user` scoped by club. Filter keystrokes do not query the DB.

Step 6 - Backend Response

The roster list is returned once per provider fetch; local filter returns a Dart list synchronously.

Step 7 - Model Conversion

Initial JSON maps become `Player` objects before `RosterFilter.apply` examines name/tier/position.

Step 8 - State Update

Search/filter state changes cause provider/widget recomputation; original loaded list remains available.

Step 9 - UI Update

The list/grid rebuilds with matching `PlayerCards`. Empty matches render the local empty state.

Call chain: roster screen load -> `squadProvider` -> GET squad -> `List<Player>` -> user types/selects -> filter state change -> `RosterFilter.apply(players,criteria)` -> filtered cards.

Failure path: initial GET failure renders provider error; an empty query/filter is not an error and returns all/applicable players. Limitation: no server pagination/scalable search.

Workflow 23: AI / Recommendation / Chatbot

`NOT IMPLEMENTED IN CURRENT REPOSITORY`

Trigger

No AI/recommendation/chatbot button, lifecycle callback, or job exists.

Step 1 - UI

No AI prompt, recommendation card, or chatbot widget.

Step 2 - State / Controller Layer

No AI provider/controller.

Step 3 - Service / Repository Layer

No inference/model API repository.

Step 4 - Backend / API

No AI endpoint or third-party AI SDK call.

Step 5 - Database

No prompt, embedding, prediction, recommendation, or model-output table.

Step 6 - Backend Response

Not applicable.

Step 7 - Model Conversion

Not applicable.

Step 8 - State Update

Not applicable.

Step 9 - UI Update

Not applicable. Player overall scores and squad progress are fixed arithmetic, not AI.

Workflow 24: Admin / Coordinator Account Operations

Trigger

Coordinator submits the portal create-account form handled by `portal.views.create_account(request)`; protected admin clients can POST `/api/admin/users/`, `/api/admin/players/`, or guardian-link endpoints.

Step 1 - UI

Portal `create_account.html` collects account type and role-specific fields. The coordinator is already session-authenticated; server derives the club. Django admin/admin API provides global operations.

Step 2 - State / Controller Layer

Django form or DRF serializer validates. There is no Flutter controller. Portal decorators and `IsAdmin` permission protect their respective entry points.

Step 3 - Service / Repository Layer

Coordinator path calls `portal.services.create_club_account(account_type, coordinator=request.user, data=cleaned_data)`. The service derives `coordinator.club`. Player creation delegates to the single `accounts.services.provision_player` aggregate service; other roles use the appropriate app/web provisioning service.

Step 4 - Backend / API

Portal POST uses Django session + CSRF + role. Mobile-user provisioning calls Firebase Admin server-side to create/link an identity; portal-only staff/coordinator uses Django password. Admin endpoints use Bearer auth plus `IsAdmin`.

Step 5 - Database

Insert/update `accounts_user` (email/names/Firebase UID/role/club), `academy_playerprofile` for players, and `accounts_guardianlink` for families. Club is the Coordinator's trusted server value or the dedicated Admin-player flow's guardian-derived same Club. Firebase temporary passwords are not stored in plaintext in these tables.

Step 6 - Backend Response

Portal success message/temporary credential handling; API serializer JSON. Duplicate email/UID, invalid role/link, Firebase failure, or DB failure produces validation/provisioning error. A newly created Firebase identity is deleted on relational failure when compensation applies.

Step 7 - Model Conversion

Portal uses Django model/form objects. Admin API uses DRF user/player/link serializers. No normal Flutter admin model flow exists.

Step 8 - State Update

Database/Firebase identities become available after successful commit; portal redirects or admin client refreshes.

Step 9 - UI Update

Coordinator roster/account lists show the new user. The user can then sign into their intended channel if active and correctly club/profile provisioned.

Call chain: create-account form -> `create_account()` -> role-specific form -> `create_club_account(coordinator=request.user)` -> server-derived Club -> `provision_player()` or role-appropriate service -> Firebase Admin where applicable -> transaction -> portal success.

Invariant: generic user creation and manual generic Admin creation exclude `PLAYER`; non-Admin member accounts require an active Club. Every Player creation path uses `provision_player`, which requires an active Club and creates exactly one `PlayerProfile` plus an optional same-Club GuardianLink atomically. `seed_users` repairs/assigns the active demo Club to every non-Admin seed and ensures its Player profile; `seed_academy` likewise repairs seeded Player/Guardian/Coach roles, Club membership, and profiles before related data is created.

Workflow 25: Guardian PIN Unlock and Child Selection

Trigger

Guardian dashboard watches `linkedPlayersProvider`; choosing a child sets `selectedChildIdProvider`. `PlayerPrivacyGate` PIN submit triggers the verification controller/provider.

Step 1 - UI

`GuardianDashboardScreen` first displays redacted `PlayerSelectorSerializer` data. The gate collects a 4-6 digit PIN. The selected `playerId` and PIN are passed to the privacy controller.

Step 2 - State / Controller Layer

Privacy providers query PIN status, enter loading for verify, call `verifyPlayerPrivacyPinProvider`, then store the returned unlock token in `PlayerUnlockTokenStore` and mark the player ID unlocked. Token store is memory-only.

Step 3 - Service / Repository Layer

`VerifyPlayerPrivacyPin.call()` -> `ApiPlayerPrivacyPinRepository.verifyPin(playerId,pin)` -> authenticated POST. After success, `selectedChildDetailsProvider` calls `GetPlayerDetails` and API repository attaches `X-Player-Unlock`.

Step 4 - Backend / API

`POST /api/players/{id}/pin/verify/` -> `PlayerPrivacyPinVerifyView.post()` checks player self or linked guardian, then `verify_pin()`. Protected detail calls `GET /api/players/{id}/profile/`, attendance/injury/eligibility endpoints and `require_player_unlock()`.

Step 5 - Database

Read/update `academy_playerprivacypin` (`pin_hash`, `failed_attempts`, `locked_until`) inside transaction/row lock; read `accounts_guardianlink`; read requested child domain tables only after authorization. Signed unlock itself is stateless and not stored in DB.

Step 6 - Backend Response

Success `{verified:true, unlockToken:<signed>}`. Invalid PIN -> 400; locked -> 423 with `lockedUntil`; missing PIN -> validation; wrong link -> 403. Detail success returns serializers; expired/mismatched token is denied.

Step 7 - Model Conversion

PIN response becomes privacy result/token value; child detail JSON becomes `Player.fromJson()`, histories become their respective entity models.

Step 8 - State Update

Token saved under player ID; unlocked-set provider changes; dependent `selectedChildDetailsProvider` refetches. Logout clears tokens; expiration/denial causes a new gate cycle.

Step 9 - UI Update

Gate is replaced by `_GuardianUnlockedContent`; child profile/stat/history cards render. Errors/lock time are shown by the PIN UI.

Call chain: select child -> privacy gate -> verify action -> privacy controller -> `VerifyPlayerPrivacyPin` -> API repository -> `PlayerPrivacyPinVerifyView.post()` -> `verify_pin()` -> `issue_player_unlock()` -> memory store -> detail GET with header -> `require_player_unlock()` + GuardianLink -> child UI.

Variable Trace: Player ID and unlock

Selector `Player.id` -> `selectedChildIdProvider` -> PIN endpoint URL -> signed payload `{user_id,player_id}` -> token store keyed by player ID -> `X-Player-Unlock` -> backend compares request user/URL player and active link.

Failure path: wrong PIN increments failures; fifth triggers 15-minute lock; expired token/detail 403 causes error/reunlock; changing URL player fails binding; removing GuardianLink makes an otherwise signed token insufficient.

Workflow 26: Injury Create / Update / Delete

Trigger

Player presses add in `InjuryHistoryScreen` (`onPressed` near line 95) or taps an existing writable record; `_InjuryFormSheet` save/delete buttons call `_save()`/`_delete()`.

Step 1 - UI

Form controls create an `InjuryRecord` draft with description/body area/status, occurred/resolved dates, and notes. Read-only coach/guardian views do not enable write callbacks.

Step 2 - State / Controller Layer

`InjuryFormController.submit(record)` or `.remove(record)` sets `AsyncLoading`, calls `saveInjuryProvider/deleteInjuryProvider`, invalidates `injuriesProvider(playerId)`, and returns saved/boolean or stores `AsyncError`.

Step 3 - Service / Repository Layer

`SaveInjury/DeleteInjury` use cases call `ApiInjuryRepository`, which POSTs for a new record, PUTs `/api/injuries/{id}/` for update, or DELETEs it; GET loads history. Bearer token and optional guardian unlock header are added.

Step 4 - Backend / API

`InjuryRecordListCreateView.post()` allows only `PLAYER` and assigns `player=request.user`. `InjuryRecordDetailView._get_record(write=True)` requires owning player for PUT/DELETE. Read paths allow owner, same-club coach, admin, or unlocked linked guardian.

Step 5 - Database

Insert/update/delete `academy_injuryrecord`: PK `id`, FK `player_id`, description/body/status/date/notes/timestamps. Player deletion cascades these rows.

Step 6 - Backend Response

Create returns 201 model JSON; update returns 200; delete 204; list returns 200 list. Validation, ownership, link/unlock, network, and not-found errors become controller/provider errors.

Step 7 - Model Conversion

JSON ↔ `InjuryRecord.toJson()`/`InjuryRecord.fromJson()`.

Step 8 - State Update

Mutation controller invalidates the player-keyed history provider; read provider refetches authoritative list.

Step 9 - UI Update

Form sheet closes on success; history list rebuilds. Failure keeps the form and shows error state/feedback.

Call chain: add/edit/delete action -> form `_save()`/`_delete()` -> `InjuryFormController.submit/remove()` -> use case -> `ApiInjuryRepository` -> injury API view -> serializer/model -> `InjuryRecord.fromJson()`/204 -> invalidate -> list.

Failure path: invalid dates/fields -> UI/server validation; guardian/coach write attempt -> 403; missing ID on delete returns false locally; network/server error -> `AsyncError`, no optimistic deletion.

Workflow 27: Dispute Raise and Response

Trigger

Coach presses submit in `FlagDisputeScreen` near line 123, invoking `_submit()`. Response action in the dispute list/detail calls `DisputeFormController.respond()`.

Step 1 - UI

Flag form holds category, summary, optional detail, and optional subject `Player.id`. `_submit()` validates required summary and calls the controller.

Step 2 - State / Controller Layer

`DisputeFormController.raise(...)/respond(...)` sets `AsyncLoading`, invokes `raiseDisputeProvider/respondToDisputeProvider`, invalidates `disputesProvider`, and returns `Dispute?` or `AsyncError`.

Step 3 - Service / Repository Layer

`RaiseDispute/RespondToDispute` -> `ApiDisputeRepository.raiseDispute()/respondToDispute()`. JSON includes category/summary/detail/subject player or response body/optional status transition.

Step 4 - Backend / API

Create: `POST /api/disputes/` -> `DisputeListCreateView.post()` requires coach and checks subject player club. Response: `POST /api/disputes/{id}/responses/` -> `DisputeResponseCreateView.post()` applies authorized dispute scope and transactionally appends response/updates status.

Step 5 - Database

Insert `academy_dispute` (raiser/subject/category/status/summary/detail/times) or `academy_disputeresponse` (dispute/author/body/status_change_to/time); response FK cascades with dispute, actor links can become null.

Step 6 - Backend Response

HTTP 201 serialized `Dispute`, including thread responses. Invalid role/club/status/body -> 400/403; missing thread -> 404; network/non-201 -> `DisputeRepositoryException`.

Step 7 - Model Conversion

Response JSON -> `Dispute.fromJson()` and nested response conversion.

Step 8 - State Update

Controller invalidates `disputesProvider`, causing `GetDisputes` to refetch the role-scoped list.

Step 9 - UI Update

Create screen closes/succeeds and dispute list/thread rebuilds; appended response/status appears.

Call chain: flag submit -> `_submit()` -> `DisputeFormController.raise()` -> `RaiseDispute` -> `ApiDisputeRepository.raiseDispute()` -> POST -> `DisputeListCreateView.post()` -> `Dispute` insert -> `Dispute.fromJson()` -> invalidate/list. Response follows controller `.respond()` -> response API -> append row/status -> updated dispute.

Failure path: empty/invalid form -> no request; cross-club subject/non-coach -> 403; invalid transition/body -> 400; controller error retains input. This is an internal dispute, **not a scouting report**.

Workflow 28: Player Session Confirmation

Trigger

In `session_confirmation_button.dart`, a player presses a confirmation option; the local `respond(status)` function calls `SessionConfirmationController.submit(session.id, playerId, status)`.

Step 1 - UI

The widget watches `sessionConfirmationsProvider(playerId)` and controller submitting set. Buttons disable while the session ID is submitting.

Step 2 - State / Controller Layer

`SessionConfirmationController.submit()` adds session ID to its `Set<String>`, calls `confirmSessionProvider(sessionId,playerId,status)`, invalidates confirmations on success, catches errors and returns false, then removes the ID. The controller prevents duplicate concurrent taps.

Step 3 - Service / Repository Layer

`ConfirmSession.call()` -> `ApiSessionConfirmationRepository.confirmSession()`. Although `playerId` crosses client layers, the API payload deliberately contains only `{sessionId,status}`.

Step 4 - Backend / API

`POST /api/session-confirmations/`; `SessionConfirmationView.post()` requires role `PLAYER`, derives player from `request.user`, validates session same club and today, and calls `update_or_create(player=request.user,session=...)`.

Step 5 - Database

Insert/update `academy_sessionconfirmation`, unique `(player_id,session_id)`, with status/responded time. Both FKs cascade on deletion.

Step 6 - Backend Response

HTTP 200/201 serialized confirmation; invalid date/session/status/role/club returns error. Repository converts non-success into an exception.

Step 7 - Model Conversion

JSON -> `SessionConfirmation.fromJson()`.

Step 8 - State Update

Provider invalidation refetches the confirmation list; the submitting set removes the session ID. The widget checks the returned boolean and surfaces a failed submission instead of silently swallowing it.

Step 9 - UI Update

On successful refetch, selected response/confirmation UI changes. On submit failure, buttons re-enable and a SnackBar tells the player that the response could not be updated. An initial confirmation-list load error renders a **Retry RSVP** button.

Call chain: response button -> local `respond()` -> `SessionConfirmationController.submit()` -> `ConfirmSession.call()` -> API repository -> POST -> `SessionConfirmationView.post()` -> `update_or_create()` -> `SessionConfirmation.fromJson()` -> invalidate/refetch -> button state.

Failure path: repository exception -> controller catches/returns false -> visible retry-oriented SnackBar; initial load error -> **Retry RSVP** button. No false optimistic confirmation is shown.

Workflow 29: Password Reset and Change

Trigger

Login "forgot password" calls `_handleForgotPassword()`; change-password screen save calls `ChangePasswordController.submit(...)`.

Step 1 - UI

Forgot flow reads trimmed email and uses login controller reset state. Change flow reads current/new/confirm password fields and provides visibility/loading/error widgets.

Step 2 - State / Controller Layer

`LoginController.sendResetEmail(email)`/`PasswordResetController.send(email)` call `sendPasswordResetProvider`. `ChangePasswordController.submit()` runs `_validate`: current required, new/confirm match, minimum eight, new differs; then calls `changePasswordProvider`.

Step 3 - Service / Repository Layer

`SendPasswordReset` -> `FirebaseAuthRepository.sendPasswordResetEmail(email)`. `ChangePassword` -> repository `changePassword`: get current user/email, build `EmailAuthProvider.credential`, `reauthenticateWithCredential`, then `user.updatePassword(next)`.

Step 4 - Backend / API

These are Firebase Auth client operations, not Django academy endpoints. Django receives the new token/session on later API calls and continues mapping the same UID.

Step 5 - Database

Firebase identity password changes outside Django ORM. Django mobile user password is not used/stored for this login path.

Step 6 - Backend Response

Firebase completes or throws typed exceptions (invalid email/current password, expired session, throttling/network). Repository maps to `AuthException` friendly messages.

Step 7 - Model Conversion

No domain JSON model; success is `void`/boolean controller outcome.

Step 8 - State Update

Controllers set sending/loading/error/success state and prevent duplicate submission.

Step 9 - UI Update

Forgot flow shows feedback that reset was requested; change flow shows success/navigates per screen or error text. Fields/buttons react to controller state.

Call chains: forgot action -> `_handleForgotPassword()` -> `LoginController.sendResetEmail()` -> `SendPasswordReset.call()` -> Firebase repository -> `FirebaseAuth.sendPasswordResetEmail()` -> feedback.

Change button -> `ChangePasswordController.submit()` -> `_validate()` -> `ChangePassword.call()` -> repository reauthenticate -> `updatePassword()` -> success/error UI.

Failure path: local change validation -> no Firebase call; wrong current password -> mapped error; no current user/email -> expired-session error; network/Firebase error -> controller error. There is no Django DB rollback because no Django write occurs.

Workflow 30: Coach Creates or Updates a Completed Match

Trigger

The Coach opens the Progress tab, taps the score icon to enter `CoachMatchesScreen`, then taps **Record Match** or edits an existing match from `MatchRosterScreen`.

Step 1 - UI

`EditFootballMatchScreen` validates opponent, competition, played date, venue, and both scores, then creates a `FootballMatchDraft`. Only completed matches are accepted; the date cannot be in the future.

Step 2 - State / Controller Layer

`MatchManagementController.create()` or `.saveMatchChanges()` enters `AsyncLoading`, invokes the appropriate use case, stores `AsyncData` on success, and invalidates `footballMatchesProvider` plus the affected match roster when editing.

Step 3 - Service / Repository Layer

`CreateFootballMatch` or `UpdateFootballMatch` calls `ApiMatchRepository.createMatch()` or `.updateMatch()`. The repository serializes the draft and uses `AuthenticatedApiClient`.

Step 4 - Backend / API

Create uses `POST /api/matches/`; update uses `PUT /api/matches/{matchId}/`. `FootballMatchListCreateView.post()` permits only `COACH`, rejects a coach without a Club, and stamps `club=request.user.club` and `created_by=request.user`. `FootballMatchDetailView.put()` calls `_coach_match()`, so the match ID must belong to the authenticated Coach's Club. The client cannot choose Club ownership.

Step 5 - Database

Create inserts `academy_footballmatch` and `academy_auditlog`; update changes only editable match metadata and adds an audit row. `FootballMatch.save()` calls `full_clean()`. The Club FK uses `PROTECT`, creator uses `SET_NULL`, and the `(club, -played_on)` index supports scoped history ordering.

Step 6 - Backend Response

Create returns HTTP 201 and update HTTP 200 with `FootballMatchSerializer` JSON. A future date, blank opponent, invalid venue, or score outside 0-99 returns 400. A wrong role or cross-Club mutation returns 403/404.

Step 7 - Model Conversion

JSON becomes `FootballMatch.fromJson()`. The Dart entity derives display helpers such as score label and win/draw/loss outcome without changing stored data.

Step 8 - State Update

The controller resolves and invalidates affected providers. The list provider refetches the Club-scoped match collection.

Step 9 - UI Update

The form closes on success; `CoachMatchesScreen` shows the new or corrected score card. Selecting a card opens `MatchRosterScreen` for per-player data entry.

Call chain: Progress score icon -> `CoachMatchesScreen` -> **Record Match** -> `EditFootballMatchScreen._submit()` -> `MatchManagementController.create()` -> `CreateFootballMatch.call()` -> `ApiMatchRepository.createMatch()` -> `POST /api/matches/` -> `FootballMatchListCreateView.post()` -> `FootballMatchSerializer.save()` with server Club/creator -> audit -> `FootballMatch.fromJson()` -> provider invalidation -> match card.

Failure path: local form validation prevents malformed submission; API validation returns a visible repository/controller error; non-Coach and cross-Club access fail before mutation; a database failure prevents both successful response and provider refresh.

Workflow 31: Coach Records or Corrects One Player's Match Performance

Trigger

The Coach opens a match card, selects a same-Club player in `MatchRosterScreen`, enters match statistics in `EditMatchPerformanceScreen`, and presses Save. The same screen can delete an existing row after confirmation.

Step 1 - UI

The form builds `MatchPerformanceDraft` from position, starter flag, minutes, goals, assists, shots, passing, defending, cards, goalkeeper fields, coach rating, and notes. Goalkeeper-only fields are shown/meaningful for position `GK`.

Step 2 - State / Controller Layer

`MatchManagementController.savePerformance(matchId, playerId, draft)` enters loading and calls `saveMatchPerformanceProvider`. Success invalidates both `matchPerformancesProvider(matchId)` and `playerMatchStatisticsProvider(playerId)`. Delete follows the same invalidation rule.

Step 3 - Service / Repository Layer

`SaveMatchPerformance.call()` delegates to `ApiMatchRepository.savePerformance()`, which sends the draft to `PUT /api/matches/{matchId}/performances/{playerId}/`. `DeleteMatchPerformance` uses DELETE on the same URL.

Step 4 - Backend / API

`MatchPerformanceDetailView.put()` first resolves the match through `_coach_match()` and resolves the player with role `PLAYER` plus `club_id=request.user.club_id`. Inside `transaction.atomic()`, it locks the parent match and existing performance row, validates the full replacement payload, checks team goal totals and goalkeeper concessions against the match score, then saves with server-owned `match`, `player`, and `recorded_by` fields.

Step 5 - Database

The mutation inserts or updates `academy_playermatchperformance` and records `academy_auditlog`. The unique `(match, player)` constraint prevents duplicates. Database checks enforce `shots_on_target <= shots`, `goals <= shots_on_target`, and `passes_completed <= passes_attempted`; model validation also protects player role/Club, position, clean sheet, card, rating, and goalkeeper rules.

Step 6 - Backend Response

The first save returns 201; correction returns 200; deletion returns 204. The read serializer returns nested match metadata and normalized camelCase statistic fields. Impossible statistics return 400; wrong role or Club scope returns 403/404.

Step 7 - Model Conversion

Response JSON becomes `MatchPerformance.fromJson()` with nested `FootballMatch`. Draft and response remain separate types so server IDs and ownership fields are not client-writable.

Step 8 - State Update

The controller resolves, then invalidates the match roster and player statistics providers. Riverpod refetches both authoritative views.

Step 9 - UI Update

The editor closes after a successful save; the match roster shows the recorded player row. The Player's summary/history and the Coach's trend screen reflect the new record after refresh.

Call chain: match card -> `MatchRosterScreen` -> player row -> `EditMatchPerformanceScreen._save()` -> `MatchManagementController.savePerformance()` -> `SaveMatchPerformance.call()` -> `ApiMatchRepository.savePerformance()` -> authenticated PUT -> `MatchPerformanceDetailView.put()` -> scoped match/player lookup -> atomic locks -> serializer/business checks -> `PlayerMatchPerformance.save()` -> audit -> `MatchPerformance.fromJson()` -> invalidate roster/player statistics.

Variable trace: `FootballMatch.id` + `Player.id` -> Riverpod/controller parameters -> URL `matchId/playerId` -> `_coach_match()` and same-Club player query -> database `(match_id, player_id)` unique key -> serialized IDs -> provider-family cache keys.

Failure path: invalid numeric relationships or goalkeeper rules return 400; a second concurrent save is serialized by row locks and the unique constraint; a player from another Club or another Club's match never reaches the save; transaction failure leaves no partial performance row.

Workflow 32: Player and Coach View Match Statistics and Trends

Trigger

A Player opens the Match Statistics tab in `ProgressScreen`; a Coach taps a player in `CoachProgressScreen` or taps **View Match Performance Trends** in `PlayerProfileScreen`.

Step 1 - UI

`PlayerMatchStatisticsView` watches `playerMatchStatisticsProvider(playerId)`. It renders a season summary, Last 5/Last 10/All rating chart, expandable match history, pull-to-refresh, retry state, or a clear empty state.

Step 2 - State / Controller Layer

The family `FutureProvider` calls `getPlayerMatchStatisticsProvider(playerId)`. Its `AsyncValue` controls loading, data, and error widgets. The history-range selector is local UI state and filters already authorized returned rows.

Step 3 - Service / Repository Layer

`GetPlayerMatchStatistics.call()` delegates to `ApiMatchRepository.fetchPlayerStatistics()`, which performs authenticated `GET /api/players/{playerId}/match-statistics/` and converts the returned map.

Step 4 - Backend / API

`PlayerMatchStatisticsView.get()` calls `_may_read_match_statistics()`: Admin may read all, Coach only same-Club players, Player only self, and Guardian/other roles are denied. Optional `from`, `to`, and `limit` query parameters are validated; limit must be 1-100.

Step 5 - Database

The endpoint reads `academy_playermatchperformance` joined to match, Club, and player, filtered by `player_id` and optional match date range, ordered newest first. `build_performance_summary()` calculates totals, pass-completion percentage, clean sheets, and one-decimal average rating from at most 100 authorized rows. It performs no write.

Step 6 - Backend Response

HTTP 200 returns `{playerId, playerName, summary, performances}`. Invalid date/limit gives 400; disallowed identity gives 403; missing player gives 404.

Step 7 - Model Conversion

`PlayerMatchStatistics.fromJson()` creates `MatchPerformanceSummary` and a typed `List<MatchPerformance>`, each containing a nested `FootballMatch`.

Step 8 - State Update

The provider resolves to `AsyncData`. Coach saves/deletes invalidate this player-specific provider; pull-to-refresh explicitly refetches it.

Step 9 - UI Update

Summary tiles show matches, goals, assists, rating, pass rate, and saves/tackles. `PerformanceTrendChart` plots chronological coach ratings; history cards expose match score, date, minutes, attack/passing/defending data, goalkeeper saves when applicable, and notes.

Call chain: Progress/player-profile trend action -> `PlayerMatchStatisticsView` -> `playerMatchStatisticsProvider(playerId)` -> `GetPlayerMatchStatistics.call()` -> `ApiMatchRepository.fetchPlayerStatistics()` -> authenticated GET -> `_may_read_match_statistics()` -> filtered `PlayerMatchPerformance` query -> `build_performance_summary()` + serializer -> `PlayerMatchStatistics.fromJson()` -> summary/chart/history.

Authorization answer: a Player cannot substitute another player ID because Django compares the URL ID to `request.user.id`; a Coach must share the target's Club; Guardians are deliberately excluded from this feature. Flutter navigation is convenience, not enforcement.

Failure path: provider error renders retry UI; empty history renders an explicit message; malformed range/limit stays a 400 rather than silently broadening access; unauthorized player/guardian/cross-Club Coach receives no statistics.

Cross-Cutting Trace: Shared Authenticated API Client and Owner-Scoped Cache

Live academy REST adapters delegate authentication, the 15-second timeout, accepted-status checks, safe server-error extraction, and multipart handling to `data/network/authenticated_api_client.dart` instead of duplicating those policies in each repository. `FirebaseAuthRepository` keeps the `/api/auth/me/` login/restore boundary separate because it creates or restores the identity that the shared client later consumes.

Successful GET chain: Riverpod/use case -> domain repository adapter -> `AuthenticatedApiClient.get(path)` -> current `ApiIdentity(uid,getIdToken)` -> Bearer request -> accepted HTTP response -> JSON validation -> `ApiGetCache.put(ownerUid,requestKey,response)` -> repository model conversion -> provider/UI.

Network-only fallback chain: token-network failure, timeout, socket, TLS handshake, or `http.ClientException` before a response -> `_networkFallback()` -> `ApiGetCache.get(currentUid,requestKey)` -> cached response marked with `x-footpath-cache: true`, if a fresh owner-scoped entry exists. An HTTP 401/403/5xx, missing identity, or malformed successful JSON throws its typed auth/HTTP/decode exception and never falls back. Requests carrying `X-Player-Unlock` are neither cached nor served from cache, and writes/multipart uploads are never cached or replayed by this client.

Attendance mutation replay remains a separate, intentionally narrow path: `OfflineFirstAttendanceRepository` queues a complete attendance batch only on `AttendanceNetworkException`; HTTP rejection remains visible. Its session-roll-call fallback likewise reads only the current owner's latest queued batch and only after a network exception. Other successful authenticated GETs may use the owner-scoped response cache, but this does not make every mutation offline-capable.

Repository Production Hardening Trace

The repository contains a production image and service topology (`backend/Dockerfile`, `compose.production.yml`) using Gunicorn, WhiteNoise, PostgreSQL, and Redis; production settings add TLS/HSTS/cookie hardening, structured logs, and optional Sentry without request PII. `/api/health/` is a cheap liveness probe, while `/api/ready/` checks database and cache dependencies and returns 503 when unavailable. CI runs deploy checks, validates the guarded PostgreSQL backup/restore scripts, and builds the backend image.

Operational flow: container start -> migrations/static collection -> Gunicorn -> liveness/readiness probes -> platform log/optional Sentry collection. Backup flow: operator/scheduler -> `scripts/postgres_backup.py` -> `pg_dump` custom-format file + retention. Restore-drill flow: explicit isolated database name -> `scripts/postgres_restore.py` confirmation guard -> `pg_restore`.

Evidence boundary: these are repeatable repository artifacts and documented procedures, not evidence of a verified live deployment, exercised monitor/alert, scheduled backup, or successful restore drill. `docs/PRODUCTION-OPERATIONS.md` deliberately leaves those evidence entries unsupplied.

Database Query Traces

QRY-1: Resolve Authenticated User

- **Source File:** `backend/accounts/authentication.py`
- **Function:** `FirebaseAuthentication.authenticate()`
- **Query:** local `User` lookup after Firebase Admin verifies decoded UID.
- **Table:** `accounts_user`.

- **Filter:** `firebase_uid=decoded['uid']`, active user; non-admin club presence checked.
- **Data Sent:** Bearer ID token only; no role/club claim is trusted from Flutter.
- **Data Returned:** Django `User` attached to `request.user`.
- **Error Handling:** invalid/missing/revoked token or local account state raises authentication failure.
- **Allowed Role:** any active configured role; admin may be clubless.

QRY-2: Read Squad

- **Source File:** `backend/academy/views.py`
- **Function:** `SquadListView.get()`.
- **Query:** `PlayerProfile.objects.select_related('user')`, coach adds `user__club=request.user.club`.
- **Table:** `academy_playerprofile` joined to `accounts_user`.
- **Filter:** same club for coach; all profiles for admin.
- **Data Sent:** no body; caller identity in token.
- **Data Returned:** `PlayerSerializer(...,many=True).data`.
- **Error Handling:** non-coach/admin raises `PermissionDenied`.
- **Allowed Role:** coach, admin.

QRY-3: Read Own/Guardian Player Profile

- **Source File:** `backend/academy/views.py`
- **Function:** `MyProfileView.get()` / `PlayerDetailView.get()`.
- **Query:** `get_object_or_404(PlayerProfile.objects.select_related('user'), user=request.user|user_id=player_id)`.
- **Table:** `academy_playerprofile`, `accounts_user`; guardian check reads `accounts_guardianlink`.
- **Filter:** player self or permitted guardian/player ID plus unlock.
- **Data Sent:** URL player ID for guardian, Bearer token, optional `X-Player-Unlock`.
- **Data Returned:** one `PlayerSerializer` map.
- **Error Handling:** 403 role/link/unlock, 404 profile.
- **Allowed Role:** player self; linked unlocked guardian through detail path; coach roster uses QRY-2.

QRY-4: Save Assessment

- **Source File:** `backend/academy/views.py`
- **Function:** `PlayerAssessmentView.put()`.
- **Query:** load profile by `user_id`; serializer update; audit insert.
- **Table:** `academy_playerprofile`, `academy_auditlog`.
- **Filter:** target player ID and equality of player/coach club IDs.
- **Data Sent:** nested 12 ratings + `coachNotes`.
- **Data Returned:** updated `PlayerSerializer` map.
- **Error Handling:** serializer 400, role/club 403, missing 404; notification after commit.
- **Allowed Role:** same-club coach.

QRY-5: Create/List Training Session

- **Source File:** `backend/academy/views.py`
- **Function:** `TrainingSessionListCreateView.get()/post()`.
- **Query:** `_sessions_for(user)` for reads; serializer `.save(created_by=user,club=user.club)` for insert.
- **Table:** `academy_trainingsession`; audit row on create.
- **Filter:** club sessions for normal user; admin helper returns all.
- **Data Sent:** title/date/time/location/tiers/focus JSON.
- **Data Returned:** serialized list or new row.
- **Error Handling:** non-coach POST 403; invalid payload 400; DB error rollback.
- **Allowed Role:** authenticated roles can read their scoped sessions; coach creates.

QRY-6: Replace Session Attendance

- **Source File:** `backend/academy/views.py`
- **Function:** `SessionAttendanceView.post()`.
- **Query:** session lookup; in-club player ID set; atomic `update_or_create`; delete omitted session rows.
- **Table:** `academy_trainingsession`, `accounts_user`, `academy_attendance`.

- **Filter:** session ID, coach club ID, submitted player IDs.
- **Data Sent:** `records[]` with player/status/optional effort/note.
- **Data Returned:** every current attendance row for the session.
- **Error Handling:** date/serializer 400; role/club/player set 403; atomic rollback.
- **Allowed Role:** same-club coach.

QRY-7: Player Attendance History

- **Source File:** `backend/academy/views.py`
- **Function:** `AttendanceListView.get()`.
- **Query:** select related session/recorder/player and filter `player_id`.
- **Table:** `academy_attendance` plus related tables.
- **Filter:** query parameter `player`; authorization/link/unlock first.
- **Data Sent:** `?player=<id>`, token, optional unlock header.
- **Data Returned:** serialized attendance list.
- **Error Handling:** missing parameter 400; unauthorized 403.
- **Allowed Role:** player self, linked unlocked guardian, same-club coach, admin according to helper.

QRY-8: Squad Progress Aggregate

- **Source File:** `backend/academy/views.py`
- **Function:** `SquadProgressView.get()`.
- **Query:** player profile list + attendance `.values('player_id').annotate(Count(...),Avg('effort'))`.
- **Table:** `academy_playerprofile`, `accounts_user`, `academy_attendance`.
- **Filter:** Coach adds `user__club_id=request.user.club_id` and `player__club_id=request.user.club_id`; Super Admin adds no Club filter.
- **Data Sent:** token only.
- **Data Returned:** manually shaped list of per-player aggregate maps.
- **Error Handling:** wrong role 403; DB failure 5xx.
- **Allowed Role:** Coach sees their own Club; Super Admin sees all Clubs.

QRY-9: Verify PIN

- **Source File:** `backend/academy/pin_service.py`, `academy/views.py`.
- **Function:** `verify_pin()` called by `PlayerPrivacyPinVerifyView.post()`.
- **Query:** transactional/select-for-update `PlayerPrivacyPin` row; GuardianLink existence before call.
- **Table:** `academy_playerprivacypin`, `accounts_guardianlink`.
- **Filter:** one player ID and current guardian/user relationship.
- **Data Sent:** PIN candidate; never stored/returned.
- **Data Returned:** verified boolean and signed unlock token; updated failure/lock state.
- **Error Handling:** invalid 400, locked 423, not linked 403, not set 400.
- **Allowed Role:** player self or linked guardian.

QRY-10: Eligibility Change and History

- **Source File:** `backend/portal/services.py`, `academy/signals.py`, `academy/views.py`.
- **Function:** `set_player_eligibility()`, `fire_eligibility_changed()`, `EligibilityHistoryView.get()`.
- **Query:** update profile status; insert history/audit; later filter history by player.
- **Table:** `academy_playerprofile`, `academy_eligibilityhistory`, `academy_auditlog`.
- **Filter:** staff/player same club on update; reader authorization/unlock on GET.
- **Data Sent:** new eligibility enum.
- **Data Returned:** portal redirect; later serialized history.
- **Error Handling:** form/permission errors; signal only logs a real change.
- **Allowed Role:** school staff updates; authorized player/guardian/staff/admin reads.

QRY-11: Injury CRUD

- **Source File:** `backend/academy/views.py`.
- **Function:** `InjuryRecordListCreateView`, `InjuryRecordDetailView`.
- **Query:** filter by caller/player; serializer create/update; model delete.
- **Table:** `academy_injuryrecord`.

- **Filter:** owning player for writes; role/club/link/unlock for reads.
- **Data Sent:** injury fields for POST/PUT.
- **Data Returned:** row/list or 204.
- **Error Handling:** validation 400, access 403, missing 404.
- **Allowed Role:** player writes own; player/coach/admin/linked unlocked guardian reads per scope.

QRY-12: Dispute and Response

- **Source File:** `backend/academy/views.py`.
- **Function:** `DisputeListCreateView.post()`, `DisputeResponseCreateView.post()`.
- **Query:** create dispute; atomic create response and optional status update.
- **Table:** `academy_dispute`, `academy_disputeresponse`.
- **Filter:** user role/club and target dispute scope.
- **Data Sent:** category/summary/detail/subject or response/status transition.
- **Data Returned:** serialized dispute/thread.
- **Error Handling:** invalid/cross-club/role rejected; transaction avoids response/status partial state.
- **Allowed Role:** coach creates; authorized coach/staff/admin respond/read according to view scope.

QRY-13: Register Device

- **Source File:** `backend/academy/views.py`.
- **Function:** `DeviceRegisterView.post()`.
- **Query:** `DeviceToken.objects.update_or_create(token=..., defaults={user,platform})`.
- **Table:** `academy_devicetoken`.
- **Filter:** globally unique token.
- **Data Sent:** token/platform; authenticated actor comes from request.
- **Data Returned:** 204 no body.
- **Error Handling:** blank token 400; auth failure 401.
- **Allowed Role:** any authenticated user.

QRY-14: Account Provisioning

- **Source File:** `backend/accounts/services.py`, `backend/portal/services.py`, `backend/academy/views.py`, and both seed commands.
- **Function:** `provision_user()`, `provision_player()`, `provision_managed_player()`, `create_club_account()`, and seed repair helpers.
- **Query:** user insert/update plus profile/link creation under transactions.
- **Table:** `accounts_user`, `academy_playerprofile`, `accounts_guardianlink`.
- **Filter:** unique email/Firebase UID; guardian/player/club checks.
- **Data Sent:** trusted role-specific form/serializer data; server club.
- **Data Returned:** user/temp-credential metadata or portal result.
- **Error Handling:** generic Player creation is rejected; missing/inactive Club or invalid same-Club guardian is rejected; DB failure rolls back the aggregate and compensation deletes a newly created Firebase identity. Seed commands repair non-Admin Club scope and Player profiles idempotently.
- **Allowed Role:** coordinator for own club or admin for protected global path.

QRY-15: Photo Path

- **Source File:** `backend/academy/storage.py`, `academy/views.py`, `portal/views.py`.
- **Function:** `upload_photo()`, `PlayerPhotoUploadView.post()`, `player_photo()`.
- **Query:** external object upload followed by profile path update; signed URL query for read.
- **Table:** `academy_playerprofile.photo_path`; external private storage object.
- **Filter:** target player ID; same-Club Coordinator in the portal, same-Club Coach through the mobile API, or Super Admin through the API.
- **Data Sent:** validated bytes/content type to storage; object path to DB.
- **Data Returned:** serialized player/success redirect; signed URL on later read.
- **Error Handling:** size/MIME/signature/config/network failures prevent path update.
- **Allowed Role:** same-Club Coach/Super Admin API; same-Club Coordinator portal.

QRY-16: Notification Inbox and Read State

- **Source File:** `backend/academy/notifications.py`, `backend/academy/views.py`.
- **Function:** `_send_to_users()`, `NotificationListView`, `NotificationUnreadCountView`, `NotificationReadView`, `NotificationReadAllView`.
- **Query:** bulk-create a recipient record before FCM; list newest 100; count unread; update one/all `read_at` values.
- **Table:** `academy_notificationrecord` plus `academy_devicetoken` for optional push fanout.

- **Filter:** active recipient IDs when creating; `user=request.user` on every inbox read/update.
- **Data Sent:** server-created event/title/body/data; authenticated request for inbox actions.
- **Data Returned:** serialized `AppNotification` contract, unread count, or updated count/read record.
- **Error Handling:** cross-user record IDs return 404; FCM unavailable leaves the persisted inbox row intact.
- **Allowed Role:** any authenticated user, strictly for their own inbox.

QRY-17: Create or Update Football Match

- **Source File:** `backend/academy/views.py`, `backend/academy/serializers.py`.
- **Function:** `FootballMatchListCreateView.post()`, `FootballMatchDetailView.put()`.
- **Query:** insert/update `FootballMatch`; insert audit row.
- **Table:** `academy_footballmatch`, `academy_auditlog`.
- **Filter:** Coach role and server-owned Club; update resolves `pk=match_id, club_id=request.user.club_id`.
- **Data Sent:** opponent, competition, played date, venue, team score; no writable Club or creator.
- **Data Returned:** `FootballMatchSerializer` map.
- **Error Handling:** future date/invalid values 400; wrong role 403; cross-Club/missing match 404.
- **Allowed Role:** same-Club Coach for writes; Coach/Admin for scoped reads.

QRY-18: Upsert Player Match Performance

- **Source File:** `backend/academy/views.py`, `backend/academy/serializers.py`, `backend/academy/models.py`.
- **Function:** `MatchPerformanceDetailView.put()`.
- **Query:** locked parent/existing-row read; aggregate other player goals; insert/update one performance; insert audit.
- **Table:** `academy_footballmatch`, `academy_playermatchperformance`, `accounts_user`, `academy_auditlog`.
- **Filter:** Coach-owned match and same-Club account with role `PLAYER`; unique `(match_id, player_id)`.
- **Data Sent:** match statistics only; match/player/recorder ownership comes from URL plus verified user.
- **Data Returned:** nested `PlayerMatchPerformanceSerializer` map.
- **Error Handling:** serializer/model/score inconsistency 400; wrong role/Club 403/404; atomic rollback and row locks protect concurrency.
- **Allowed Role:** same-Club Coach.

QRY-19: Read Player Match Summary and History

- **Source File:** `backend/academy/views.py`, `backend/academy/match_statistics.py`.
- **Function:** `PlayerMatchStatisticsView.get()`, `build_performance_summary()`.
- **Query:** select related match/Club/player, filter by player and optional date range, newest-first slice of 1-100 rows.
- **Table:** `academy_playermatchperformance`, `academy_footballmatch`, `accounts_user`, `accounts_club`.
- **Filter:** player self, same-Club Coach, or Admin; Guardians are excluded.
- **Data Sent:** URL player ID; optional `from`, `to`, and `limit`.
- **Data Returned:** player identity, aggregate summary, and serialized historical rows.
- **Error Handling:** malformed/reversed date range or invalid limit 400; disallowed reader 403; missing Player 404.
- **Allowed Role:** Player self, same-Club Coach, Admin.

Variable Traces

Variable: User ID / Firebase UID

`FirebaseAuth.currentUser.uid` -> signed ID token -> Firebase Admin decoded `uid` -> `accounts_user.firebase_uid` -> Django `request.user.id` -> model actor/player foreign keys. The numeric Django ID, not Firebase UID, is used for relational player references.

Variable: Role

`accounts_user.role` -> `UserSerializer` role wire value -> `UserProfile.fromJson()` -> `HomeScreen` portal choice. For APIs the role is read again from `request.user.role`; Flutter never sends an authoritative role.

Variable: Player ID

`PlayerSerializer.id` (Django user PK) -> `Player.fromJson().id` string -> route/provider family parameter -> URL `<player_id>` or request record `playerId` -> backend same-club/ownership/link lookup -> `PlayerProfile.user_id`/related FKs.

Variable: Coach ID

Verified token UID -> local coach `request.user.id` -> server-assigned `created_by_id`, `recorded_by_id`, `actor_id`, or `raised_by_id`. It is not accepted as a trusted client payload field.

Variable: Training ID

`TrainingSession.id` returned by Django PK -> schedule card entity -> edit/delete URL or attendance/session-confirmation payload -> `TrainingSession.objects.get(pk=...)` -> attendance/confirmation `session_id` FK -> response `sessionId`.

Variable: Attendance Status

UI enum -> `_marks[playerId]` -> `Attendance.status` -> uppercase `toJson()` -> DRF choice validation -> `academy_attendance.status` -> serializer JSON -> enum `fromWire()` -> chip/aggregate.

Variable: Performance Score

Slider/input integer -> `PlayerRatings` field -> nested JSON -> `AssessmentSerializer` 0-99 validation -> `PlayerProfile` integer column -> nested response -> `PlayerRatings` -> arithmetic `overall` -> card/radar.

Variable: Match ID and Player Match Key

`FootballMatchSerializer.id` -> `FootballMatch.id` -> match-card route -> provider/controller `matchId` -> `/api/matches/{matchId}/...` -> same-Club `FootballMatch` lookup -> `PlayerMatchPerformance.match_id`. Combined with the selected `Player.id`, the database unique `(match_id, player_id)` key identifies exactly one historical performance.

Variable: Coach Rating and Trend

Form decimal 0.0-10.0 -> `MatchPerformanceDraft.coachRating` -> JSON `coachRating` -> serializer Decimal validation -> `academy_playermatchperformance.coach_rating` -> summary one-decimal mean and serialized row -> Dart `double` -> `PerformanceTrendChart` chronological point.

Variable: Academic Eligibility

Portal form enum -> cleaned value -> `set_player_eligibility` -> current profile column + signal old/new rows -> JSON history/current profile -> Dart eligibility enum/change object -> badge/card.

Variable: Guardian Unlock Token

Successful PIN candidate -> `verify_pin` -> `issue_player_unlock(request.user.id, player.id)` -> response string -> `PlayerUnlockTokenStore[playerId]` -> `X-Player-Unlock` -> `require_player_unlock()` signed user/player/max-age check.

Variable: Evaluation ID

There is no assessment/evaluation entity ID. Evaluations overwrite the player profile row keyed by player ID. Per-session effort belongs to an attendance row ID server-side, but the Dart entity identifies it through player/session rather than exposing an attendance PK.

Variable: Scout ID / Scouting Report ID

`NOT IMPLEMENTED IN CURRENT REPOSITORY`. No variable, role, model, endpoint, or table carries these IDs.

Object / Model Traces

Object: UserProfile

- **Created from:** `/api/auth/me/ UserSerializer` JSON.
- **Converted using:** `UserProfile.fromJson()`.
- **Stored in:** local widget route/bootstrap state, passed to `HomeScreen`/coach portal; Firebase maintains identity session separately.
- **Consumed by:** role routing, coach header/profile, logout flows.
- **Key fields:** local identity, display name/email, role, club.

Object: Player

- **Created from:** `PlayerSerializer` JSON from squad/me/detail/assessment/position/photo.
- **Converted using:** `Player.fromJson()` with nested rating/tier/position/eligibility converters.
- **Stored in:** Riverpod `FutureProvider` results and route constructor arguments.
- **Consumed by:** roster cards, profile, assessment editor, guardian/player dashboards.
- **Key fields:** ID/name/age/class/tier/position/12 ratings/eligibility/photo/notes.

Object: TrainingSession

- **Created from:** schedule form draft or API JSON.
- **Converted using:** `TrainingSession.toJson()/fromJson()`.
- **Stored in:** `academy_trainingsession`; transient Riverpod session list.
- **Consumed by:** schedule cards, edit form, attendance screen, player confirmation.
- **Key fields:** ID/title/tiers/date/time/location/focus/attendee count.

Object: Attendance

- **Created from:** attendance marks or API JSON.
- **Converted using:** `Attendance.toJson()/fromJson()`.

- **Stored in:** `academy_attendance`; queued batch JSON in device `outbox_attendance` on network failure.
- **Consumed by:** log screen, histories, dashboard summaries/streak, progress aggregate.
- **Key fields:** player/session/status/updated time/coach/effort/note.

Object: FootballMatch

- **Created from:** Coach `FootballMatchDraft` or `FootballMatchSerializer` JSON.
- **Converted using:** `FootballMatchDraft.toJson()` and `FootballMatch.fromJson()`.
- **Stored in:** `academy_footballmatch`; transient `footballMatchesProvider` list and route arguments.
- **Consumed by:** Coach match cards, match editor, match roster, nested performance history.
- **Key fields:** ID, opponent, competition, played date, venue, both scores; Club/creator remain server-owned.

Object: MatchPerformance

- **Created from:** Coach `MatchPerformanceDraft` or `PlayerMatchPerformanceSerializer` JSON.
- **Converted using:** draft `toJson()` and `MatchPerformance.fromJson()` with nested `FootballMatch`.
- **Stored in:** unique `academy_playermatchperformance` row; transient match/player family providers.
- **Consumed by:** Coach match roster/editor and Player/Coach summary, trend chart, and match-history cards.
- **Key fields:** player/match IDs, position/start/minutes, attacking/passing/defending/card/goalkeeper statistics, coach rating, and notes.

Object: PlayerMatchStatistics

- **Created from:** `/api/players/{playerId}/match-statistics/` aggregate response.
- **Converted using:** `PlayerMatchStatistics.fromJson()` with `MatchPerformanceSummary.fromJson()` and performance-list conversion.
- **Stored in:** Riverpod `playerMatchStatisticsProvider(playerId)` only; the summary is derived, not persisted.
- **Consumed by:** Player Match Statistics tab and Coach player-trend routes.
- **Key fields:** authorized player identity, totals/rates/average, and historical performance rows.

Object: EligibilityChange

- **Created from:** eligibility signal-created backend row returned by API.
- **Converted using:** `EligibilityChange.fromJson()`.

- **Stored in:** `academy_eligibilityhistory`; Riverpod family provider list.
- **Consumed by:** `EligibilityHistoryScreen._ChangeCard`.
- **Key fields:** old/new status, changer representation, timestamp.

Object: InjuryRecord

- **Created from:** player form draft or API JSON.
- **Converted using:** `InjuryRecord.toJson()/fromJson()`.
- **Stored in:** `academy_injuryrecord`; family provider lists.
- **Consumed by:** injury form/history for player and read-only authorized roles.

Object: Dispute

- **Created from:** coach flag form or API list/thread response.
- **Converted using:** `Dispute.fromJson()`.
- **Stored in:** `academy_dispute` with `academy_disputeresponse` children.
- **Consumed by:** dispute list/thread/form providers.

Object: SessionConfirmation

- **Created from:** player response API JSON.
- **Converted using:** `SessionConfirmation.fromJson()`.
- **Stored in:** unique `academy_sessionconfirmation` player/session row.
- **Consumed by:** `SessionConfirmationButton` and coach confirmation views/providers.

Object: AppNotification

- **Created from:** current-user `NotificationRecordSerializer` JSON.
- **Converted using:** `AppNotification.fromJson()`.
- **Stored in:** `academy_notificationrecord`; transient Riverpod inbox lists and unread count.
- **Consumed by:** `NotificationBell`, `NotificationInboxScreen`, `NotificationNavigationController`, foreground/opened-push handling, role-aware destination routing, and mark-read actions.
- **Key fields:** ID/type/title/body/neutral data/read state/timestamp; FCM is a delivery channel, not the history store.

Button-to-Database/API Traces

#	Button/action	Callback	Controller/use case	API/service	Store/result	UI change
1	Login	`LoginScreen._handleSignIn()`	`LoginController.signIn` -> `SignIn`	Firebase sign-in + `GET /api/auth/me/`	reads `accounts_user`	`HomeScreen` or error
2	Forgot password	`_handleForgotPassword()`	reset controller/use case	Firebase `sendPasswordResetEmail`	Firebase identity service	feedback/error
3	Logout	role screen `_signOut()`	`UnregisterDevice` -> `SignOut`	current-token `DELETE /api/devices/` -> Firebase `signOut`	scoped device row removed best-effort; local session/cache/unlock cleared	login route even if unregister fails
4	Save session	`ScheduleSessionScreen._submit()`	schedule controller/use case	`POST /api/training-sessions/`	insert session/audit	pop + refreshed list
5	Edit session	same `_submit()` with `existing`	`.saveChanges` -> update use case	`PUT /api/training-sessions/{id}/`	update session/audit	pop + refresh
6	Cancel session	`_cancelSession()`	controller `.cancel`	`DELETE /api/training-sessions/{id}/`	delete session; on-commit cancellation inbox/push; attendance retains null FK	list refresh
7	Finalize attendance	`LogAttendanceScreen._finalize()`	attendance controller/use case	POST attendance or local outbox	upsert/prune attendance / queue	pop + history refresh
8	Save assessment	`EditPerformanceDataScreen._save()`	assessment controller/use case	`PUT /api/players/{id}/assessment/`	update player profile/audit	updated card/profile
9	Assign position	position picker selection	position controller/use case	`PUT /api/players/{id}/position/`	update profile/audit	position label refresh
10	Verify PIN	privacy gate submit	PIN controller/use case	`POST /api/players/{id}/pin/verify/`	PIN counter/lock; signed token	unlocked child content
11	Confirm session	confirmation option `respond()`	confirmation controller/use case	`POST /api/session-confirmations/`	upsert confirmation	selected response or visible retry feedback
12	Add injury	injury form `_save()`	injury controller/use case	`POST /api/injuries/`	insert injury	refreshed history
13	Edit injury	injury form `_save()`	same	`PUT /api/injuries/{id}/`	update injury	refreshed history
14	Delete injury	injury form `_delete()`	controller `.remove`	`DELETE /api/injuries/{id}/`	delete injury	list item removed after refetch
15	Flag dispute	`FlagDisputeScreen._submit()`	dispute controller/use case	`POST /api/disputes/`	insert dispute	list/thread refresh
16	Respond dispute	response submit	controller `.respond`	`POST /api/disputes/{id}/responses/`	insert response/update status	thread refresh
17	Staff eligibility save	portal form submit	portal service	Django `set_player_eligibility`	profile + history + audit	portal/mobile refresh
18	Upload photo	Coach picker or portal multipart submit	photo controller/use case or portal service	Django multipart -> Supabase Storage REST	object + profile path	immediate/refetched avatar or visible error

#	Button/action	Callback	Controller/use case	API/service	Store/result	UI change
19	Coordinator create account	portal form submit	`create_club_account`	Firebase Admin/service/ORM	user/profile/link	roster/account list
20	Notification bell / row / push open	bell or `_openMessage()` / `_openNotification()`	notification providers + `NotificationNavigationController`	current-user notification and profile APIs	read/update `NotificationRecord`; protected destination data remains API-scoped	badged inbox or role-aware schedule/profile/eligibility destination
21	Record match	`EditFootballMatchScreen._submit()`	`MatchManagementController.create -> CreateFootballMatch`	`POST /api/matches/`	insert Club-owned match/audit	refreshed match cards
22	Save player match statistics	`EditMatchPerformanceScreen._save()`	controller -> `SaveMatchPerformance`	`PUT /api/matches/{matchId}/performances/{playerId}/`	atomic upsert unique match/player row + audit	roster and player trends refresh
23	Delete player match statistics	editor delete confirmation	controller -> `DeleteMatchPerformance`	DELETE same performance URL	delete scoped row + audit	roster and player trends refresh

Screen-to-Screen Navigation Traces

- App launch -> `MaterialApp.home` -> `SessionBootstrapScreen` -> conditional `LoginScreen`/`HomeScreen` by widget build.
- Login -> `Navigator.pushReplacement(MaterialPageRoute(HomeScreen(profile)))` -> role portal.
- `HomeScreen` -> direct widget return `CoachPortalScreen`, `PlayerPortalScreen`, or `GuardianPortalScreen`.
- Portal bottom navigation -> local tab index -> `IndexedStack` child; no route push and state is retained.
- Training list add -> `_openScheduleForm()` -> `Navigator.push(MaterialPageRoute(ScheduleSessionScreen))`; boolean result triggers refresh behavior.
- Training card tap -> `_logAttendance(session)` -> `Navigator.push(LogAttendanceScreen(session,...))`; full session object is passed.
- Player card -> `Navigator.push` player-profile screen; `Player` object/ID passed.
- Player profile edit assessment -> `Navigator.push(EditPerformanceDataScreen(player))`; saved `Player?` returns through pop.
- Coach Progress score icon -> `Navigator.push(CoachMatchesScreen)` -> **Record Match** opens `EditFootballMatchScreen`; tapping a match opens `MatchRosterScreen`; selecting a player opens `EditMatchPerformanceScreen`.
- Coach Progress player card or profile trend button -> `Navigator.push(PlayerMatchStatisticsScreen(playerId,...))`.
- Player Progress tab -> embedded `PlayerMatchStatisticsView(playerId: currentPlayer.id)`; Guardian Progress intentionally keeps training feedback only.
- Eligibility tile -> `Navigator.push(EligibilityHistoryScreen(playerId,...))`.
- Attendance history button -> `Navigator.push(AttendanceHistoryScreen(playerId,...))`.
- Injury history tile -> `Navigator.push(InjuryHistoryScreen(playerId,readOnly...))`.

- Notification bell -> `Navigator.push(NotificationInboxScreen())`. A known inbox row, foreground SnackBar **View**, `onMessageOpenedApp`, or `getInitialMessage()` -> `NotificationNavigationController` -> current profile/role resolution -> schedule tab (`session_*`), Player/authorized Guardian profile (`assessment_saved`), or Player/authorized Guardian eligibility history (`eligibility_changed`). Unknown events or profile-resolution failures open/focus the inbox. Player IDs cannot be overridden by payload data, Guardian IDs are checked against linked children, and protected destinations retain PIN/privacy gates.
- Sign out -> navigation replacement/removal to `LoginScreen`.

Navigation uses Flutter `Navigator` with `MaterialPageRoute`, plus `IndexedStack` for tabs. No GoRouter, AutoRoute, GetX, or named-route table was found.

Async Code Traces

Firebase/app bootstrap

`main() async` awaits Firebase plugin setup before constructing repositories that depend on it. Without `await`, the widget tree could request Auth/Messaging before the default app exists. The UI thread is not synchronously blocked; Flutter's event loop resumes the future.

Login and restore

Async operations wait for auth state, credential sign-in, token issuance, network HTTP, and JSON results. Controllers set loading before `await` and state/error after. Removing `await` would navigate before profile authorization or let exceptions escape the intended catch.

API read providers

Riverpod `FutureProvider` runs HTTP asynchronously and exposes loading/data/error. Widgets rebuild on completion instead of blocking frames. Live repository reads cross `AuthenticatedApiClient`; only a pre-response network failure may resolve from a fresh current-owner cache entry. Disposing a screen releases `autoDispose` provider state, although an underlying HTTP call is not automatically a database transaction cancellation.

Mutations

Async controllers disable save buttons while awaiting to reduce duplicate writes. `mounted`/`context.mounted` checks prevent navigation or `setState` on disposed UI after completion.

Attendance synchronization

Network and sqflite operations are awaited sequentially so queue order is preserved. Running all retries in parallel would break last-write intent for multiple batches targeting a session.

Unawaited device registration

Login/bootstrap uses `unawaited(registerDeviceProvider...)` because push registration is secondary and should not delay portal entry. The tradeoff is reduced immediate error visibility.

Foreground and opened notifications

`FirebaseMessaging.onMessage` and `onMessageOpenedApp` stream subscriptions live for the app state lifetime; `getInitialMessage()` handles a push that launched the app, and `onTokenRefresh` re-registers the current token. Handlers invalidate inbox/count providers and use navigator/messenger keys without requiring a screen-local context. Open actions pass through the same tested destination controller as inbox-row taps, while unknown/unsafe requests fall back to the persistent inbox.

Django `transaction.on_commit`

This callback is asynchronous in event timing, not a Flutter `Future`: it defers notification persistence/FCM side effects until the relational commit succeeds. Session cancellation snapshots recipient IDs before deleting, but registers its notification only after `session.delete()` succeeds; a failed delete cannot announce a cancellation. Notification helpers must not change the already returned domain result.

Defense answers

- **Why async/await?** Firebase, HTTP, sqflite, and platform operations finish later; `await` preserves readable ordered control flow and targeted error handling.
- **Does it block the UI?** Await yields control to Dart's event loop; rendering continues unless CPU-heavy synchronous work is performed, which these flows do not do.
- **What if `await` is removed?** Code receives a `Future` rather than its value, loading flags may reset early, navigation can race, and errors may bypass the catch.

Failure / Error Path Matrix

Workflow	Validation failure	Auth/authorization	Network/database/external failure	UI/result
Restore	malformed profile -> parse/auth error	401/403 signs out	network/5xx -> restore error	retry/continue/login, never cached role authorization
Login	empty not prechecked; Firebase rejects	local missing/inactive/clubless rejects	Firebase/HTTP error	controller error text, button re-enabled
Schedule	empty/tier/date or paired/ordered time validation	non-coach/cross-club edit 403	HTTP/DB error	field/API error; form stays + SnackBar
Attendance	unmarked/dialog/window/serializer	non-coach/cross-club/player set 403	network queues; HTTP/DB does not	queued/success pop; other error SnackBar
Assessment	malformed/range serializer	non-coach/cross-club 403	transport/DB error	form remains, old current values
Match record	blank/future/venue/score invalid	only Coach writes; match is server-Club scoped	transport/DB error	form remains; list unchanged
Match performance	inconsistent shots/goals/passes/GK/team score	only same-Club Coach writes	atomic lock/constraint/DB error	editor remains; no partial row; providers stay authoritative
Match statistics read	bad date range/limit	Player self, same-Club Coach, Admin only; Guardian denied	network/DB error	empty/retry/error state; no broadened data
Profile read	optional photo null is valid	role/link/unlock rejects	network-only may use current-owner GET cache; HTTP/DB errors remain visible	cached data or AsyncError/retry; avatar fallback
PIN	format/incorrect -> 400	missing link 403	transaction error	failure count/lock feedback; no child detail
Eligibility	invalid form/choice	staff/club/read permission	DB/notification failure	DB success survives push failure; refresh required
Confirmation	invalid today/status/session	non-player/cross-club	network/server	submit SnackBar or **Retry RSVP**; no false selected response
Injury	invalid fields/dates	only owner writes; scoped reads	network/DB	controller error, list unchanged
Dispute	summary/category/status invalid	role/club/scope	network/transaction	controller error, thread unchanged
Photo	missing/size/MIME/signature	same-Club Coordinator portal; same-Club Coach/Super Admin API	storage config/network/DB	no path update; visible error/fallback
Provisioning	duplicate/invalid role/link	coordinator/admin gate	Firebase/DB	compensation attempts Firebase cleanup
Notification	malformed inbox action/missing token/unknown event	every inbox query is current-user scoped; current profile + linked-child/privacy gates protect destinations	FCM invalid/unavailable, profile/inbox network error	business + inbox record remain; known safe route or focused-inbox fallback; device push may be absent

Authorization Trace

```
Firebase email/password or restored user
  -> Firebase ID token
  -> Authorization: Bearer
  -> Firebase Admin verify(check_revoked=True)
  -> decoded uid
  -> accounts_user(firebase_uid, active)
  -> reject non-admin without club
  -> request.user.role check
  -> queryset/object club/owner check
  -> GuardianLink check where relevant
  -> PIN unlock signature/user/player/10-minute check where relevant
  -> serializer/business validation
  -> ORM query/mutation
```

Access control is a combination of backend authentication, business/view authorization, and relational filters. Flutter role routing is UX only. There is no direct client database access and no RLS layer. The backend prevents a player from pretending to be a coach because the verified UID maps to the immutable-for-request local user role; a client-supplied role is ignored, and coach endpoints compare `request.user.role` to `Roles.COACH`.

Match-performance authorization adds two explicit boundaries. Write endpoints derive the Coach from the verified token, resolve the match only inside that Coach's Club, resolve the player only as a same-Club `PLAYER`, and never accept writable ownership fields. The statistics read endpoint separately permits only Admin, same-Club Coach, or the Player whose URL ID equals `request.user.id`; Guardian profile access does not imply match-statistics access.

Data Lifecycle Traces

1. User / Account

- **Create:** coordinator/admin form -> provisioning service -> Firebase identity where applicable + `accounts_user`.
- **Validate:** form/serializer, unique email/UID, allowed role, club rules.
- **Store:** Firebase identity and Django user; a Player is never a bare generic user-the dedicated aggregate also creates exactly one profile and optional same-Club guardian link. Seed commands repair the same Club/profile invariant for demo data.
- **Retrieve:** `/api/auth/me/`, portal/admin lists; token UID mapping.
- **Display:** role portal/profile/account lists.
- **Update:** admin role/status/account operations; password handled by Firebase for mobile or Django for web.
- **Delete/archive:** admin detail/delete behavior and Firebase linking require coordinated care; no soft-delete lifecycle was found.

2. Player Profile / Performance

- **Create:** trusted player provisioning creates `User` + one-to-one `PlayerProfile` + optional link.

- **Validate:** birth/tier/defaults and account rules; assessments 0-99 through serializer.
- **Store:** current fields in `academy_playerprofile`.
- **Retrieve:** squad/me/detail endpoints -> `Player`.
- **Display:** cards, dashboard, radar, notes, eligibility.
- **Update:** Coach assessment/position/mobile photo; School Staff eligibility; Coordinator portal photo; Super Admin photo.
- **Delete/archive:** player user deletion cascades profile/dependents; no assessment archive/history exists.

2A. Football Match / Historical Performance

- **Create:** Coach records a completed match; Django stamps Club/creator. Coach then upserts one performance per same-Club Player.
- **Validate:** match date/venue/scores; statistic relationships, goalkeeper fields, rating, Club/player role, team-goal and opponent-score consistency.
- **Store:** `academy_footballmatch` plus unique historical `academy_playermatchperformance` rows and audit events.
- **Retrieve:** Coach/Admin match roster endpoints; Player self/same-Club Coach/Admin statistics endpoint with optional date/limit filter.
- **Display:** Coach match cards and roster; Player/Coach totals, performance-rating trend chart, and match history.
- **Update:** same-Club Coach corrects match metadata or replaces a Player's complete row; providers refetch affected views.
- **Delete/archive:** same-Club Coach can delete a performance row; deleting a match cascades its rows at model level, but the mobile API exposes no match-delete endpoint.

3. Training Session

- **Create:** coach form/API -> session insert with server club/creator.
- **Validate:** required fields/choices/tiers/date plus paired 12-hour start/end values with start strictly before end, enforced by serializer and model.
- **Store:** `academy_trainingsession`.
- **Retrieve:** club-scoped training-session GET -> `TrainingSession` list.
- **Display:** schedule cards/tabs and player confirmation.
- **Update:** same-club coach PUT.
- **Delete/archive:** coach DELETE; confirmations cascade; attendance survives with null session; no soft archive.

4. Attendance

- **Create:** coach finalizes complete batch; upsert unique player/session; offline network failure queues locally.
- **Validate:** UI marks/window plus server role/club/date/serializer.

- **Store:** `academy_attendance`; temporary local `outbox_attendance` if unsent.
- **Retrieve:** session/player endpoints and progress aggregate.
- **Display:** chips, histories, streaks, percentages, effort cards.
- **Update:** re-finalizing batch updates same unique rows and prunes omissions.
- **Delete/archive:** omitted rows deleted; session deletion preserves rows with null session; successful sync deletes outbox copy.

5. Eligibility

- **Create:** first status is part of player profile; every real change creates history.
- **Validate:** portal choice, school-staff role, same club.
- **Store:** current `academy_playerprofile.eligibility` + append-only `academy_eligibilityhistory`.
- **Retrieve:** player profile and history endpoint.
- **Display:** badge/history cards.
- **Update:** school-staff portal save, signals/audit/FCM.
- **Delete/archive:** player deletion cascades history; no normal history edit/delete UI.

6. Notification

- **Create:** committed business event -> `notify_*` -> `_send_to_users()` -> one `NotificationRecord` for each active recipient, followed by optional FCM fanout.
- **Validate/scope:** event payload is server-created; recipient IDs come from Club/player/guardian relationships rather than a client-selected inbox owner.
- **Store:** authoritative history/read state in `academy_notificationrecord`; optional delivery endpoints in `academy_devicetoken`.
- **Retrieve:** current-user list and unread-count endpoints -> `AppNotification` list/count providers.
- **Display:** foreground SnackBar, badged bell, and retryable inbox; known opened pushes/rows route to the role-appropriate schedule, profile, or eligibility destination, while unknown events remain in the inbox.
- **Update:** current-user mark-one/mark-all endpoints set `read_at`; cross-user IDs are not addressable.
- **Delete/archive:** deleting a user cascades their records; there is no normal inbox delete action.

Scouting Report Lifecycle

`NOT IMPLEMENTED IN CURRENT REPOSITORY` at every lifecycle stage.

Top 23 Important Function Call Chains

1. **Startup:** `main()` -> `Firebase.initializeApp()` -> `runApp(ProviderScope)` -> `_FootPathAppState.initState()` messaging listeners -> `FootPathApp.build()` -> `SessionBootstrapScreen`.
2. **Restore:** `initState()` -> `_restore()` -> `RestoreSession.call()` -> `FirebaseAuthRepository.restoreSession()` -> `authStateChanges().first` -> `/api/auth/me/` -> `UserProfile.fromJson()` -> `HomeScreen`.
3. **Login:** `_handleSignIn()` -> `LoginController.signIn()` -> `SignIn.call()` -> `signInAndFetchProfile()` -> Firebase sign-in -> `/api/auth/me/` -> role portal.
4. **Logout:** `_signOut()` -> `UnregisterDevice.call()` -> current FCM token -> scoped `DELETE /api/devices/` (best-effort) -> `SignOut.call()` -> Firebase sign-out + current-owner GET-cache clear -> clear unlocks -> `LoginScreen`.
5. **Role authorization:** API call -> Bearer token -> `FirebaseAuthentication.authenticate()` -> local `User` -> view role check -> club/object/link check -> response/403.
6. **Player self profile:** `PlayerDashboardScreen` -> `myProfileProvider` -> `GetMyProfile.call()` -> `fetchMyProfile()` -> `MyProfileView.get()` -> `PlayerSerializer` -> `Player.fromJson()`.
7. **Guardian child profile:** select child -> PIN verify controller -> API verify -> `verify_pin()` -> `issue_player_unlock()` -> token store -> `fetchPlayerDetails()` with header -> guarded detail view -> `Player.fromJson()`.
8. **Squad list:** coach screen -> `squadProvider` -> `GetSquad.call()` -> `fetchSquad()` -> `SquadListView.get()` -> club query -> `List<Player>`.
9. **Position:** picker -> `PlayerPositionController` -> `SavePlayerPosition.call()` -> `savePosition()` -> `PlayerPositionView.put()` -> serializer save/audit -> `Player.fromJson()` -> invalidate.
10. **Assessment:** save button -> `_save()` -> `EditPerformanceController.submit()` -> `SavePlayerAssessment.call()` -> `saveAssessment()` -> `PlayerAssessmentView.put()` -> profile/audit/on-commit inbox + FCM -> `Player.fromJson()`.
11. **Create session:** add -> form `_submit()` -> `ScheduleSessionController.submit()` -> `ScheduleTrainingSession.call()` -> `createSession()` -> `TrainingSessionListCreateView.post()` -> session/audit/on-commit inbox + FCM -> refresh.
12. **Edit session:** form `_submit()` -> controller `.saveChanges()` -> `UpdateTrainingSession.call()` -> repository `updateSession()` -> `TrainingSessionDetailView.put()` -> refresh.
13. **Cancel session:** `_cancelSession()` -> controller `.cancel()` -> `CancelTrainingSession.call()` -> repository `deleteSession()` -> `TrainingSessionDetailView.delete()` -> snapshot recipients/audit/delete -> on-commit inbox + FCM -> refresh.
14. **Attendance online:** `_finalize()` -> `AttendanceLogController.save()` -> `LogSessionAttendance.call()` -> offline decorator -> API repository POST -> `SessionAttendanceView.post()` -> atomic upsert/prune -> decode/invalidate.
15. **Attendance offline replay:** API transport exception -> decorator -> `AttendanceOutbox.enqueue()` -> `AttendanceSyncService` owner query -> API repository retry -> backend save -> outbox delete.
16. **Squad progress:** progress screen -> `squadProgressProvider` -> `GetSquadProgress.call()` -> API progress repository -> `SquadProgressView.get()` -> Coach Club filter or Super Admin all-club query -> ORM Count/Avg -> `PlayerProgress.fromJson()`.

17. **Eligibility change:** staff form -> `staff_eligibility()` -> `set_player_eligibility()` -> `PlayerProfile.save()` -> pre/post signals -> history/audit -> on-commit inbox + FCM.
18. **Injury create:** injury form `_save()` -> `InjuryFormController.submit()` -> `SaveInjury.call()` -> API injury repository POST -> `InjuryRecordListCreateView.post()` -> model -> `InjuryRecord.fromJson()` -> invalidate.
19. **Dispute response:** response action -> `DisputeFormController.respond()` -> `RespondToDispute.call()` -> API POST responses -> `DisputeResponseCreateView.post()` -> response/status transaction -> `Dispute.fromJson()`.
20. **Notification delivery/open:** mutation commit -> `NotificationRecord` persistence + best-effort FCM -> foreground/opened handler, bell, or inbox row -> notification providers/API + `NotificationNavigationController` -> current-profile role mapping -> protected schedule/profile/eligibility destination or focused inbox fallback. Device registration supplies the optional FCM delivery token.
21. **Create match:** Progress score icon -> `CoachMatchesScreen` -> match form -> `MatchManagementController.create()` -> `CreateFootballMatch` -> `ApiMatchRepository.createMatch()` -> `FootballMatchListCreateView.post()` -> server Club/creator + audit -> `FootballMatch.fromJson()` -> refresh.
22. **Save match performance:** roster player -> performance editor -> controller -> `SaveMatchPerformance` -> API PUT -> `_coach_match()` + same-Club Player -> atomic locks/validation/upsert + audit -> `MatchPerformance.fromJson()` -> invalidate roster/statistics.
23. **Read match trends:** Player tab or Coach player route -> `playerMatchStatisticsProvider` -> `GetPlayerMatchStatistics` -> API GET -> `_may_read_match_statistics()` -> scoped historical query -> `build_performance_summary()` -> typed summary/history -> chart/cards.

Code-Tracing Defense Questions (38)

T1. What happens after the user presses Login?

`_handleSignIn` reads controllers, calls `LoginController.signIn`, then `SignIn` and `FirebaseAuthRepository`; Firebase authenticates, Django `/api/auth/me/` authorizes, JSON becomes `UserProfile`, and `HomeScreen` routes by role. Failure updates controller error.

T2. Which file handles authentication?

Client orchestration is `firebase_auth_repository.dart`; trusted API authentication is `backend/accounts/authentication.py`. The former obtains tokens; the latter verifies them and maps a local user.

T3. Where is the returned user session stored?

Firebase Auth's SDK maintains the mobile identity session. The Django profile is a `UserProfile` passed through bootstrap/login routes, not a custom database token stored by Flutter. Guardian unlock tokens are separate and memory-only.

T4. How is the user role obtained?

Django loads `accounts_user` by verified Firebase UID, `UserSerializer` returns its role, and `UserProfile.fromJson` parses it. Backend views re-read `request.user.role` for every authorization decision.

T5. How does the application decide which screen to show?

`HomeScreen.build()` switches on `UserProfile.role`: coach/player/guardian portals, otherwise a generic authenticated placeholder.

T6. Trace a scouting report from UI to database.

`NOT IMPLEMENTED IN CURRENT REPOSITORY`: no role, screen, controller, endpoint, model, migration, or table. The dispute flow is distinct and must not be relabeled.

T7. Trace attendance from button click to database.

Finalize -> `_finalize` builds records -> attendance controller/use case -> offline decorator -> API repository POST -> `SessionAttendanceView.post` validates coach/club/date/players -> atomic `update_or_create`/prune in `academy_attendance` -> serialized records -> Dart models/provider refresh. Network-only failure queues locally.

T8. How is player performance retrieved?

Player/squad provider -> get-player use case -> `ApiPlayerRepository` -> player endpoint -> `PlayerProfile` query/`PlayerSerializer` -> `Player.fromJson`/`PlayerRatings.fromJson` -> Riverpod data -> card/radar.

T9. Where does JSON become a Dart object?

At entity factories such as `UserProfile.fromJson`, `Player.fromJson`, `TrainingSession.fromJson`, `Attendance.fromJson`, `PlayerProgress.fromJson`, and others, called by concrete API repositories.

T10. How does database data reach a Flutter widget?

ORM queryset/model -> DRF serializer -> HTTP JSON -> API repository decode/entity factory -> use case future -> Riverpod `FutureProvider` `AsyncData` -> `ConsumerWidget` rebuild.

T11. What happens if the database request fails?

Django returns a non-success/connection closes; repository throws a domain-specific exception; controller or `FutureProvider` becomes error and UI shows retry/error. Only a network-classified attendance write is queued optimistically.

T12. Why is the login function asynchronous?

Firebase credential validation, token creation, and HTTP profile lookup complete later. Await keeps them ordered and lets loading/error state bracket the whole operation without blocking UI rendering.

T13. What data is passed between assessment functions?

Selected `Player.id`, a `PlayerRatings` object, and `coachNotes`; repository transforms them into nested ratings JSON. Coach/club/actor are derived server-side, not passed as trusted values.

T14. Why does `EditPerformanceController` exist?

It owns mutation loading/error, calls the use case, invalidates squad state, and keeps widgets independent of HTTP/Firebase mechanics.

T15. Why doesn't the UI query the database directly?

It would expose infrastructure/policy, duplicate authorization, and bypass Django transactions/audit. Repository/API and Django boundaries preserve security and replaceability.

T16. Where is validation performed?

Convenience checks occur in Flutter forms; authoritative checks occur in DRF serializers/views, Django forms/services, and some model/database constraints.

T17. Where is authorization performed?

Primarily in Django: Firebase authentication class, permission classes, role comparisons, club-filtered querysets, ownership, GuardianLink, and signed PIN unlock. UI routing is not trusted.

T18. How does the backend know which user made the request?

It verifies the Firebase Bearer token, extracts UID, finds the active `accounts_user` with that `firebase_uid`, and sets `request.user`.

T19. What happens when the user logs out?

The screen first calls `UnregisterDevice`: it obtains the current FCM token and sends a current-user-scoped `DELETE /api/devices/`; absence, timeout, or HTTP failure is logged and cannot block logout. `SignOut` then calls `FirebaseAuthRepository._signOutAndClearCache()`: it captures the owner UID, signs out of Firebase, clears that owner's durable API GET cache, clears guardian/player unlock memory, and replaces the authenticated portal with login. Only the matching device-token row is deleted; no training/player domain row is removed.

T20. How are sessions restored after app restart?

`SessionBootstrapScreen.initState` calls restore; repository waits for Firebase auth state, gets a fresh ID token, and calls `/api/auth/me/`. Only a valid local profile reaches `HomeScreen`.

T21. How does offline attendance reach the database later?

Outbox stores owner/session/record JSON. Sync loads current-owner rows oldest first, converts JSON back to `Attendance`, calls the same live repository, and removes only after server success.

T22. What stops offline data crossing accounts?

Each outbox row has `owner_uid`; current sync requests rows for only the active Firebase UID. The general GET cache uses the same owner UID in its composite key and is cleared for that owner on sign-out; unlock-protected reads are excluded entirely.

T23. What happens if the attendance API returns 403 while offline logic is enabled?

It becomes a general attendance repository exception, not `AttendanceNetworkException`, so it is not queued. The controller shows failure.

T24. How does an assessment return to the exact screen?

Backend returns updated player JSON; repository creates `Player`; controller returns it to `_save`; screen `Navigator.pop(saved)` supplies it to the previous route and invalidates the squad for a refetch.

T25. How is the guardian unlock token produced and consumed?

Successful transactional PIN verification calls `issue_player_unlock(user,player)`; Flutter stores returned token under player ID; detail repositories add `X-Player-Unlock`; backend verifies signature, age, binding, and link.

T26. What is stored in the PIN table?

Player one-to-one key, `pin_hash`, failed-attempt count, lock-until timestamp, and update time-not plaintext PIN and not guardian unlock tokens.

T27. How does eligibility update reach mobile?

Staff portal saves current profile; signals add history/audit and send FCM on commit. Mobile does not receive a state stream; its provider refetches profile/history on open/refresh/invalidation conditions.

T28. How is progress generated?

One endpoint groups attendance by player and annotates status counts and average effort, combines it with player profiles, returns JSON, and Flutter maps to `PlayerProgress` cards.

T29. How does file upload avoid exposing a Supabase service key?

Browser or Flutter sends multipart to Django; the Flutter path uses `AuthenticatedApiClient.postMultipart()` and never receives the storage credential. Django validates bytes, calls Supabase REST using server environment credentials, stores only the object path, and returns signed read URLs.

T30. What happens when a session is canceled?

Coach confirms UI -> delete use case/repository -> same-club `TrainingSessionDetailView.delete` -> snapshot recipients and audit -> model delete -> register on-commit cancellation inbox/FCM -> session list invalidation. Attendance session FK becomes null; confirmations cascade. If deletion fails, the on-commit callback is never registered/executed as a successful cancellation.

T31. Why can the player ID be trusted in a URL?

It is not trusted by itself. Django combines it with verified `request.user`, role, club/ownership/link, and optional unlock checks before querying/returning data.

T32. Where is a new account's club assigned?

Coordinator portal passes the authenticated Coordinator's Club into `create_club_account`/provisioning. Generic Admin user creation requires an active selected Club and excludes `PLAYER`; the dedicated Admin-player path derives the Player Club from the required active same-Club guardian and calls `provision_player`. Seed commands also repair every non-Admin seed's Club and ensure each seeded Player has exactly one profile.

T33. How is a duplicate RSVP prevented?

Backend uses `update_or_create` on the database-unique `(player,session)` pair. A new response updates rather than stacks another row.

T34. What happens while a `FutureProvider` is waiting?

Riverpod exposes loading; the `ConsumerWidget` renders a progress state. On resolution it stores data/error and rebuilds only consumers of that provider.

T35. Where is notification receiving traced?

Backend event helpers persist current-user `NotificationRecord` rows before optional FCM. Flutter traces `onMessage` to a foreground SnackBar and provider invalidation, and traces its **View** action plus `onMessageOpenedApp`, `getInitialMessage()`, and inbox-row taps through `NotificationNavigationController`. The controller re-resolves `/api/auth/me/`, suppresses duplicate active opens, marks the matching row read best-effort, and maps session events to Schedule, assessments to the Player/authorized Guardian child Profile, and eligibility changes to the protected eligibility history; unknown/unsafe requests fall back to the inbox. Local tests verify the inbox/router policy, but signed physical-device FCM/APNs delivery remains environment-dependent and unverified.

T36. Who enters match-performance data?

The Coach is the normal encoder. Django verifies `request.user.role == COACH`, derives the Club/recorder from that identity, and restricts both match and Player to the same Club. Players receive read-only access to their own history; Admin can inspect but cannot create through these endpoints.

T37. How are impossible match statistics rejected?

The write serializer and model check field ranges and relationships such as goals <= shots on target <= shots and completed passes <= attempted passes. Goalkeeper-only fields require `GK`; clean sheet conflicts with goals conceded; the transactional view also rejects combined Player goals above the team score and goalkeeper concessions above the opponent score. Database constraints provide a final guard for key numeric relationships and duplicate match/player rows.

T38. How can a Player see trends without reading another Player's data?

Flutter sends the selected/current Player ID, but Django does not trust it. `_may_read_match_statistics()` compares a Player URL ID with `request.user.id`, checks same-Club scope for a Coach, permits Admin, and rejects Guardians/other roles. Only after authorization does it query rows and build the summary.

Memorization Cards - Twelve Critical Workflows

Card 1: Login

What I click

Login button.

First function called

`LoginScreen._handleSignIn()`.

Next function

`LoginController.signIn()` -> `SignIn.call()`.

Service used

`FirebaseAuthRepository.signInAndFetchProfile()`.

Database/API operation

Firebase email/password; then Bearer `GET /api/auth/me/`; `accounts_user` UID/active/club lookup.

Data returned

Serialized local user profile.

State change

`LoginState` loading/error; profile returned; device/sync start.

Screen result

`HomeScreen` and role portal.

20-second defense explanation

Firebase authenticates the credentials, but Django then verifies the token and returns the active local role/club profile. Flutter parses it and routes to the correct portal.

60-second technical explanation

The button reads trimmed email and raw password into `LoginController`. It sets loading, calls `SignIn`, then `FirebaseAuthRepository`, which signs in through Firebase, obtains an ID token, and calls `/api/auth/me/`. Django verifies revocation, maps UID to an active user, enforces club assignment, and serializes the profile. Dart converts it to `UserProfile`; login starts device registration/sync and replaces the route. Any Firebase/API exception becomes controller error text.

Card 2: Session Restoration

What I click

Nothing; app startup triggers it.

First function called

`SessionBootstrapScreen.initState()` -> `_restore()`.

Next function

`RestoreSession.call()`.

Service used

`FirebaseAuthRepository.restoreSession()`.

Database/API operation

Firebase auth-state read + `GET /api/auth/me/` local user query.

Data returned

`UserProfile?`.

State change

Bootstrap profile/completed/error local state.

Screen result

Home, login, or restore error.

20-second defense explanation

The app never trusts a cached role. It restores Firebase identity, obtains a token, and rechecks the Django account before showing an authenticated portal.

60-second technical explanation

`initState` calls `_restore`, which invokes the restore use case. The repository awaits the first Firebase auth state. No user yields login. A user yields a fresh ID token and `/api/auth/me/`; Django checks revocation, active local UID mapping, and club. JSON becomes `UserProfile`. The screen sets local state, starts device registration/attendance sync, and builds `HomeScreen`. A 401/403 signs out; recoverable network failure shows retry.

Card 3: Player Profile Retrieval

What I click

Open player dashboard/profile.

First function called

Widget watches `myProfileProvider` or related player provider.

Next function

`GetMyProfile.call()` / `GetPlayerDetails.call()`.

Service used

`ApiPlayerRepository.fetchMyProfile()` / `fetchPlayerDetails()`.

Database/API operation

GET player endpoint; read `academy_playerprofile` joined to `accounts_user`.

Data returned

`PlayerSerializer` JSON.

State change

`FutureProvider` loading -> data/error.

Screen result

Player card, ratings, notes, eligibility, photo.

20-second defense explanation

The provider calls a player use case and authenticated API repository. Django applies role/object rules, serializes the current profile, and Riverpod rebuilds the dashboard from a typed `Player`.

60-second technical explanation

The player dashboard watches `myProfileProvider`, which invokes `GetMyProfile` and the API repository. The shared client sends a Bearer GET to `/api/players/me/`; Django requires player role and queries the profile related to `request.user`. Guardians use the detail route plus unlock header, which disables persistent caching for that request. `PlayerSerializer` returns nested ratings and optional photo URL; `Player.fromJson` creates enums/nested `PlayerRatings`; the provider changes from loading to data and the consumer renders cards. A pre-response network failure may use the current owner's cached successful GET; HTTP/auth/decode errors become `AsyncError`.

Card 4: Guardian Unlock

What I click

Select child and submit PIN.

First function called

Privacy gate verification callback/controller.

Next function

`VerifyPlayerPrivacyPin.call()`.

Service used

`ApiPlayerPrivacyPinRepository.verifyPin()`.

Database/API operation

POST PIN verify; row-locked PIN hash/failure state + GuardianLink read; signed token issuance.

Data returned

`verified` and `unlockToken`.

State change

Token stored in memory for player ID; unlocked provider changes.

Screen result

Full child content fetch/render.

20-second defense explanation

A valid guardian login and link are not enough for private detail. The server verifies the hashed household PIN, throttles guesses, and issues a ten-minute token bound to guardian and child.

60-second technical explanation

The redacted selector supplies player ID. The gate posts the PIN with Firebase auth. Django verifies the current GuardianLink, locks the PIN row, checks lock time and hash, updates failures, then signs a user/player payload. Flutter keeps it only in memory and adds it as `X-Player-Unlock` to detail/history requests. The server rechecks signature age/binding and the current link. Five failures lock for 15 minutes; a different player ID or expired token fails.

Card 5: Create Training Session

What I click

Add session, then Save.

First function called

`ScheduleSessionScreen._submit()`.

Next function

`ScheduleSessionController.submit()` -> `ScheduleTrainingSession.call()`.

Service used

`ApiTrainingRepository.createSession()`.

Database/API operation

`POST /api/training-sessions/`; insert `academy_trainingsession` with server club/creator.

Data returned

Serialized `TrainingSession`.

State change

Controller loading/data/error; session provider refreshed.

Screen result

Form pops and session card appears.

20-second defense explanation

The form builds a typed session, the controller/use case posts it, and Django validates coach role then assigns the trusted club and creator before saving and returning it.

60-second technical explanation

`_submit` validates required fields and tiers, constructs `TrainingSession`, and calls the async controller. The use case delegates to `ApiTrainingRepository`, which sends `toJson` with Firebase Bearer token. `TrainingSessionListCreateView.post` requires coach, validates the serializer, saves with `created_by=request.user` and `club=request.user.club`, audits, and schedules FCM on commit. Response becomes `TrainingSession.fromJson`; state resolves and the route pops/refetches. Errors keep the form and show a SnackBar.

Card 6: Attendance Online/Offline

What I click

Finalize Attendance.

First function called

`LogAttendanceScreen._finalize()`.

Next function

`AttendanceLogController.save()` -> `LogSessionAttendance.call()`.

Service used

`OfflineFirstAttendanceRepository`, then `ApiAttendanceRepository` or `AttendanceOutbox`.

Database/API operation

POST complete batch -> atomic attendance upsert/prune; network failure -> local outbox insert.

Data returned

Saved records or optimistic queued records.

State change

Controller resolves; provider invalidates; queue sync state persists when offline.

Screen result

Success/queued pop; other errors remain visible.

20-second defense explanation

The coach submits the complete marked roster. Django validates and replaces the session attendance atomically. A network-only failure stores the owner-scoped batch locally and replays it later in order.

60-second technical explanation

`_finalize` validates the time window/unmarked players and builds typed records, clearing effort/note for non-present. Controller/use case calls an offline decorator that tries the API. Django checks coach, same-club session, 0-2-day window, record schema, and every player club; a transaction upserts submitted rows and deletes omissions. JSON returns to `Attendance.fromJson` and providers refresh. If transport fails, the decorator serializes the full batch with Firebase UID in sqflite. Sync replays oldest first, deletes success, and stops with retry metadata on failure.

Card 7: Assessment Save

What I click

Save on performance editor.

First function called

`EditPerformanceDataScreen._save()`.

Next function

`EditPerformanceController.submit()` -> `SavePlayerAssessment.call()`.

Service used

`ApiPlayerRepository.saveAssessment()`.

Database/API operation

PUT assessment; update 12 rating columns and notes in `academy_playerprofile`; audit/push.

Data returned

Updated `Player` JSON.

State change

Controller returns `Player`; squad invalidated.

Screen result

Editor pops; updated rating/card appears.

20-second defense explanation

The coach sends 12 ratings and notes; Django verifies same-club coach access, validates 0-99, updates the current profile, audits, and returns a typed player.

60-second technical explanation

The form builds `PlayerRatings` and notes, controller enters loading, and `SavePlayerAssessment` delegates to the API repository. The PUT carries nested ratings and no trusted actor/club. Django's view requires coach, compares club IDs, validates/ flattens through `AssessmentSerializer`, saves the profile, records audit, and schedules notification after commit. `PlayerSerializer` response becomes `Player.fromJson`; Riverpod invalidates squad and the editor returns the saved object. Current data overwrites; no assessment-history row exists.

Card 8: Performance Dashboard

What I click

Coach Progress tab.

First function called

`CoachProgressScreen` watches `squadProgressProvider`.

Next function

`GetSquadProgress.call()`.

Service used

`ApiProgressRepository.fetchSquadProgress()`.

Database/API operation

GET progress; ORM groups attendance and computes Count/Avg.

Data returned

List of `PlayerProgress` maps.

State change

FutureProvider loading/data/error.

Screen result

Squad summary and per-player cards.

20-second defense explanation

Opening the tab triggers one authenticated aggregate request. Django computes attendance counts and average effort per club player, and Flutter converts the list into progress cards.

60-second technical explanation

`CoachProgressScreen` consumes `squadProgressProvider`. The get-progress use case calls its API adapter through `AuthenticatedApiClient`. `SquadProgressView` checks Coach/Super Admin, Club-filters both profile and attendance querysets only for a Coach, and leaves them all-club for Super Admin before the filtered `Count/Avg` group-by. It shapes a JSON list with zero/null defaults. `PlayerProgress.fromJson` converts each row; Riverpod resolves and rebuilds summary/card widgets. This is deterministic analytics, not AI.

Card 9: Eligibility Update and Read

What I click

Staff submits eligibility; player/guardian opens history.

First function called

Portal `staff_eligibility()`; read screen watches `eligibilityHistoryProvider`.

Next function

`set_player_eligibility()`; read `GetEligibilityHistory.call()`.

Service used

Portal service/signals; `ApiEligibilityHistoryRepository` for read.

Database/API operation

Update profile, insert eligibility history/audit; later GET history.

Data returned

Portal success and serialized `EligibilityChange` list.

State change

Server current/history state; mobile FutureProvider refetch.

Screen result

Updated badge/history cards.

20-second defense explanation

School staff changes a same-club eligibility status. Signals preserve old/new history and audit after the profile save; authorized player or unlocked guardian later retrieves the history.

60-second technical explanation

The CSRF/session-protected staff form validates a club player and status, then calls `set_player_eligibility`. Saving `PlayerProfile` triggers `pre_save` to stash the prior value and `post_save` to insert `EligibilityHistory`, audit, and on-commit FCM only when changed. Mobile history uses a player-keyed Riverpod provider and authenticated endpoint; Django checks role/link/unlock, serializes rows, and Dart maps them to `EligibilityChange` cards. Grades are never stored.

Card 10: Injury Create

What I click

Add Injury, then Save.

First function called

`_InjuryFormSheetState._save()`.

Next function

`InjuryFormController.submit()` -> `SaveInjury.call()`.

Service used

`ApiInjuryRepository`.

Database/API operation

`POST /api/injuries/`; insert player-owned `academy_injuryrecord`.

Data returned

Serialized `InjuryRecord`.

State change

Controller loading/data/error; player injury provider invalidated.

Screen result

Form closes and history list refreshes.

20-second defense explanation

The player submits a typed injury form; Django derives the owner from the authenticated player, validates and inserts the record, then Flutter refetches the history.

60-second technical explanation

The form creates an `InjuryRecord` draft and calls `InjuryFormController`, which invokes `SaveInjury` and the API repository. The repository delegates the authenticated POST to `AuthenticatedApiClient`. The view requires player role and ignores any attempt to choose another owner by saving `player=request.user`. Serializer writes the row and returns JSON; `InjuryRecord.fromJson` converts it, controller invalidates `injuriesProvider(playerId)`, and the list rebuilds. Invalid fields, network errors, and non-owner writes remain errors with no optimistic mutation.

Card 11: Record Match and Player Performance

What I click

Coach Progress score icon -> Record Match -> match card -> Player -> Save.

First function called

`EditFootballMatchScreen._submit()`, then `EditMatchPerformanceScreen._save()` for the Player row.

Next function

`MatchManagementController.create()` / `.savePerformance()` -> corresponding use case.

Service used

`ApiMatchRepository` through `AuthenticatedApiClient`.

Database/API operation

`POST /api/matches/`, then `PUT /api/matches/{matchId}/performances/{playerId}/`; insert Club-owned match and atomic unique Player performance.

Data returned

Typed `FootballMatch`, then typed `MatchPerformance` with nested match data.

State change

Controller loading/data/error; match list, roster, and affected Player statistics providers invalidated.

Screen result

New score card and recorded roster row; trends become available to the Coach and Player.

20-second defense explanation

The Coach records a completed match and one row per Player. Django derives ownership from the verified Coach, validates Club and statistic consistency, saves atomically, audits, and returns typed data for provider refresh.

60-second technical explanation

The match form creates a draft and posts it through the controller/use case/repository chain. Django accepts only a Coach, stamps Club and creator, validates the completed date and score, and saves an audit row. Selecting a Player sends match/player IDs in the URL plus statistics in the body. Django re-resolves both IDs inside the Coach's Club, locks the parent and existing row, validates statistic relationships and team-score totals, then inserts or replaces the unique historical row with `recorded_by=request.user`. The controller invalidates roster and Player-statistics providers so both views refetch authoritative data.

Card 12: View Match Statistics and Trends

What I click

Player Progress -> Match Statistics, or Coach -> Player -> View Match Performance Trends.

First function called

`PlayerMatchStatisticsView.build()` watches `playerMatchStatisticsProvider(playerId)`.

Next function

`GetPlayerMatchStatistics.call()`.

Service used

`ApiMatchRepository.fetchPlayerStatistics()`.

Database/API operation

`GET /api/players/{playerId}/match-statistics/`; authorized historical query plus derived summary.

Data returned

`PlayerMatchStatistics` containing `MatchPerformanceSummary` and chronological history rows.

State change

Family FutureProvider loading -> data/error; pull-to-refresh refetches.

Screen result

Season summary tiles, Last 5/10/All rating chart, and expandable match history.

20-second defense explanation

Django permits only the Player themself, a same-Club Coach, or Admin, then summarizes historical match rows. Flutter converts the response into totals, a rating trend, and match details.

60-second technical explanation

The view watches a Player-keyed FutureProvider, which invokes the use case and authenticated repository GET. The backend checks the verified user's role and relationship to the URL Player before querying. It optionally validates date and limit filters, loads at most 100 newest historical rows, calculates pass rate, totals, clean sheets, and one-decimal average rating, then serializes summary plus history. Dart constructs nested typed objects; Riverpod resolves, and the UI renders summary tiles, a chronological coach-rating chart, and expandable match cards. Unauthorized IDs return 403 and never enter the query/aggregation path.

Final Trace Verification Notes

- Every connected chain above was checked against the referenced screen/provider/use-case/repository/view/model files.
- Table names follow Django defaults verified from model/migration naming; `PlayerEligibility` is a proxy and has no table.
- No secret values, tokens, database passwords, or service credentials are included.
- Scout/scouting and AI paths are marked `NOT IMPLEMENTED IN CURRENT REPOSITORY` because no executable connection exists.
- Notification persistence, current-user inbox/read APIs, device registration/unregister, foreground SnackBar handling, bell navigation, duplicate suppression, best-effort read marking, and role-aware session/profile/eligibility destinations are traceable and locally tested. Real Firebase/APNs delivery and presentation on a signed physical device are not claimed as verified.
- Container/compose, health/readiness, optional Sentry, backup/restore scripts, runbook, and CI image-build paths are traceable repository artifacts; no live deployment, executed alert, scheduled backup, or successful restore drill is claimed.
- The code comment in `PlayerAssessmentView` still says "six ratings," while the executable entity/profile supports twelve (six outfield plus six goalkeeper); this trace follows the actual fields.
- Match-performance tracing reflects commit `db9e869`: secure Club-owned match creation, atomic per-Player historical statistics, Player/self and same-Club Coach reads, derived summaries/trends, Flutter screens/providers, and focused/full regression results (271 Django tests, 250 Flutter tests, clean Flutter analysis, no migration drift).